

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE  
POLITEHNICA BUCUREȘTI  
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE  
DEPARTAMENTUL DE CALCULATOARE



## PROIECT DE DIPLOMĂ

Politică de securitate adaptivă  
Sistem adaptiv de gestionare a politicilor de securitate pentru  
serve Linux

Gabriel-Claudiu Țîntu

**Coordonator științific:**

Prof. dr. ing. Răzvan Victor Rughiniș

**BUCUREȘTI**

2026

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY  
POLITEHNICA BUCHAREST  
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS  
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



## DIPLOMA PROJECT

Adaptive security policy  
Adaptive security policy management system for Linux servers

Gabriel-Claudiu Țîntu

**Thesis advisor:**

Prof. dr. ing. Răzvan Victor Rughiniș

**BUCHAREST**

2026

# CUPRINS

|          |                                   |           |
|----------|-----------------------------------|-----------|
| <b>1</b> | <b>Introducere</b>                | <b>1</b>  |
| 1.1      | Context . . . . .                 | 1         |
| 1.2      | Problema . . . . .                | 1         |
| 1.3      | Obiective . . . . .               | 2         |
| 1.4      | Soluția propusă . . . . .         | 3         |
| 1.5      | Rezultatele obținute . . . . .    | 3         |
| 1.6      | Structura lucrării . . . . .      | 4         |
| <b>2</b> | <b>Analiza cerințelor</b>         | <b>5</b>  |
| 2.1      | Utilizatorul vizat . . . . .      | 5         |
| 2.2      | Scenarii de utilizare . . . . .   | 5         |
| 2.3      | Cerințe funcționale . . . . .     | 6         |
| 2.4      | Cerințe non-funcționale . . . . . | 7         |
| 2.5      | Cerințe de sistem . . . . .       | 8         |
| 2.6      | Constrângeri . . . . .            | 8         |
| <b>3</b> | <b>Studiu de piață</b>            | <b>10</b> |
| 3.1      | Soluții existente . . . . .       | 10        |
| 3.1.1    | Fail2ban . . . . .                | 10        |
| 3.1.2    | CrowdSec . . . . .                | 11        |
| 3.1.3    | Wazuh . . . . .                   | 12        |
| 3.1.4    | Alte soluții . . . . .            | 13        |
| 3.2      | Oportunitatea . . . . .           | 14        |
| 3.3      | Tehnologiile utilizate . . . . .  | 14        |
| 3.3.1    | Backend . . . . .                 | 15        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 3.3.2    | Frontend . . . . .                                    | 15        |
| 3.3.3    | Bază de date . . . . .                                | 15        |
| 3.3.4    | Autentificare și identitate . . . . .                 | 15        |
| 3.3.5    | Containerizare . . . . .                              | 16        |
| 3.3.6    | Aplicare . . . . .                                    | 16        |
| 3.3.7    | Asistență . . . . .                                   | 16        |
| <b>4</b> | <b>Soluția propusă</b>                                | <b>17</b> |
| 4.1      | Vedere în ansamblu asupra arhitecturii . . . . .      | 17        |
| 4.1.1    | Nivelul de prezentare (interfața web) . . . . .       | 18        |
| 4.1.2    | Nivelul de logică (serverul aplicației) . . . . .     | 18        |
| 4.1.3    | Nivelul de execuție (gazda Linux) . . . . .           | 18        |
| 4.1.4    | Servicii externe . . . . .                            | 19        |
| 4.1.5    | Acces securizat prin port knocking . . . . .          | 19        |
| 4.2      | Fluxul adaptiv . . . . .                              | 19        |
| 4.3      | Scenariu de utilizare . . . . .                       | 20        |
| <b>5</b> | <b>Detalii de implementare</b>                        | <b>22</b> |
| 5.1      | Organizarea codului . . . . .                         | 22        |
| 5.2      | Motorul de detecție . . . . .                         | 23        |
| 5.3      | Planificatorul adaptiv și logica de decizie . . . . . | 26        |
| 5.4      | Aplicarea măsurilor pe gazdă . . . . .                | 27        |
| 5.5      | Port Knocking . . . . .                               | 28        |
| 5.6      | Politica de securitate configurabilă . . . . .        | 28        |
| 5.7      | Baza de date . . . . .                                | 29        |
| 5.8      | Autentificare și controlul accesului . . . . .        | 30        |
| 5.9      | Notificări în timp real . . . . .                     | 31        |
| 5.10     | Monitorizarea rețelei . . . . .                       | 31        |
| 5.11     | Asistentul AI . . . . .                               | 32        |
| 5.12     | Interfața web și pagina de analiză . . . . .          | 33        |

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>6</b> | <b>Evaluare</b>                          | <b>34</b> |
| 6.1      | Verificarea funcționalităților . . . . . | 34        |
| 6.2      | Evaluarea performanțelor . . . . .       | 35        |
| 6.3      | Discuție . . . . .                       | 36        |
| <b>7</b> | <b>Concluzii</b>                         | <b>38</b> |
| 7.1      | Dezvoltări ulterioare . . . . .          | 38        |
|          | <b>Bibliografie</b>                      | <b>40</b> |
|          | <b>Anexe</b>                             | <b>41</b> |
|          | Diagrame . . . . .                       | 41        |

## **SINOPSIS**

Această lucrare prezintă proiectarea și implementarea unui sistem de securitate adaptiv pentru servere Linux, care automatizează apărarea împotriva atacurilor uzuale, păstrând administratorul în procesul decizional. Față de un firewall obișnuit, cu reguli fixe, sistemul reacționează în funcție de comportamentul observat, colectând evenimentele din sistem și din conexiunile active, pe care le combină într-o evaluare unică a fiecărei adrese și le blochează pe cele periculoase în baza unei politici de securitate configurabile în timpul rulării. Interfața web a aplicației oferă o viziune asupra întregii activități de pe sistemul gazdă, prin alerte în timp real, un istoric al operațiilor executate, lista cu reguli firewall, conexiuni active și statistici. Validat pe un mediu Linux, sistemul demonstrează capacitatea de a opri prompt atacurile uzuale, confirmând că o apărare automatizată poate rămâne eficientă și transparentă.

## **ABSTRACT**

This thesis presents the design and implementation of an adaptive security system for Linux servers, which automates defense against common attacks, keeping the administrator in the decision-making process. Compared to a regular firewall, with fixed rules, the system reacts based on the observed behavior, collecting events from the system and from active connections, which it combines into a unique evaluation of each address and blocks the dangerous ones based on a security policy configurable at runtime. The application's web interface provides a view of all activity on the host system, through real-time alerts, a history of executed actions, the list of firewall rules, active connections and statistics. Validated on a Linux environment, the system demonstrates the ability to promptly stop common attacks, confirming automated defense can remain efficient and transparent.

## MULȚUMIRI

*Mulțumesc domnului profesor coordonator Răzvan Victor Rughiniș, pentru îndrumarea oferită în realizarea proiectului de licență.*

# 1 INTRODUCERE

## 1.1 Context

Serverele Linux găzduiesc o sumedenie de servicii expuse în internet, fie că vorbim de aplicații web, de interfețe de programare (API - Application Programming Interface) sau infrastructuri de administrare accesate de la distanță prin SSH (Secure Shell). Indiferent de forma sub care sunt găzduite, în cloud public, pe un server privat (VPS) sau pe o mașină administrată personal, aceste servicii sunt vizibile și, implicit, accesibile oricui. În realitate, odată ce un astfel de server este conectat la internet, devine ținta unui flux de trafic periculos. Cercetările bazate pe sisteme capcană (honeypot) arată că un serviciu expus este descoperit și atacat în decurs de doar câteva minute de la conectare [11], iar unele experimente au raportat o medie de aproximativ 70 de atacuri pe minut asupra unor sisteme expuse [10]. Majoritar, aceste incidente nu reprezintă atacuri țintite, ci mai degrabă acțiuni oportuniste, programele automate scanând continuu spațiul adreselor IP (Internet Protocol) publice în căutarea de servicii accesibile pe care să le compromită la scară largă [11].

Una dintre frecventele forme de atac împotriva serverelor Linux este forța brută (brute force) asupra serviciului SSH [21], unde atacatorul încearcă repetat perechi utilizator-parolă, conturile vizate fiind, în general, cele cu privilegii ridicate. Utilizatorul principal (root) reprezintă peste o treime din toate încercările de autentificare observate în astfel de studii, precum cel realizat de Cisco [12], iar datorită automatizării masive, suita de atacuri este executată foarte rapid. Dincolo de atacurile de tip brute force, serverele sunt constant vizate de scanări ale porturilor, încercări de autentificare cu utilizatori inexistenți, precum și de inițierea unui val mare de conexiuni simultane, menite să suprasolicite și să epuizeze resursele sistemului.

Prin urmare, securizarea unui server expus nu mai este o sarcină pe care administratorul o poate trata ocazional, atacurile fiind permanente, automate și la un cost redus pentru atacator, în timp ce apărarea rămâne, în multe cazuri, manuală și lentă. Acest dezechilibru justifică necesitatea unor mecanisme de apărare capabile să analizeze comportamentul traficului și să reacționeze automat, în timp real.

## 1.2 Problema

Sistemul de apărare de bază a unui server Linux se bazează pe un firewall configurat manual, ale cărui reguli rămân fixe odată setate. Un astfel de firewall fiind, prin natura lui, static, el aplică exact regulile scrise de administrator și nu observă, nici nu reacționează, la compor-



tamentul celor care încearcă să acceseze serverul. Administratorul serverului are în întregime responsabilitatea de a urmări jurnalele sistemului, de a recunoaște tipare suspecte și de a bloca manual adresele cu comportament agresiv.

Regulile fixe impun o alegere incomodă, serviciul SSH fiind expus continuu la încercări de brute force, dacă acesta rămâne accesibil public, în timp ce închiderea lui completă respinge chiar și accesul legitim. Neținând cont de context, un filtru static nu oferă o cale de mijloc între aceste extreme. Acesta nu poate tolera câteva autentificări eșuate izolate și, totodată, să reacționeze la un atac susținut, singura cale de ajustare fiind reconfigurarea manuală a regulilor.

La această rigiditate se adaugă o serie de neajunsuri practice. De la inițierea unui atac și intervenția unui administrator trece un interval de timp în care atacatorul poate acționa liber. Acest dezavantaj este amplificat de limitele monitorizării umane, deoarece atacurile se desfășoară fără întrerupere, lăsând sistemul vulnerabil în perioadele nesupravegheate. Totuși, vulnerabilitatea persistă chiar și atunci când sistemul este sub supraveghere activă, întrucât volumul mare de date generat de un server cu trafic ridicat face imposibilă detectarea manuală a anomaliilor. La rândul lor, instrumentele clasice îngreunează acest efort, tratând fiecare alertă izolat și ratând riscul cumulat pe care îl prezintă o sursă în timp. Pentru că atacatorii își schimbă frecvent adresele IP, majoritatea surselor agresive fiind active o singură zi [11], strategia de blocare manuală devine un efort fără sfârșit.

Intervenția umană directă este greu de urmărit în timp și lipsită de nuanțe, iar o adresă este fie blocată complet, fie lăsată liberă, fără trepte intermediare de avertizare pe măsură ce semnele de comportament malițios se acumulează. Deciziile luate pe loc nu lasă în urmă un istoric structurat, așa că, după un timp, motivul și contextul blocării unei anumite surse devin greu de reconstituit, complicând astfel atât auditarea, cât și corectarea acțiunilor eronate.

Problema centrală abordată în această lucrare este, astfel, absența unei legături automate între detectarea unui comportament suspect și reacția la acesta. Eliminarea acestor neajunsuri ale modelului tradițional de apărare presupune un sistem care să înlăture această ruptură, capabil să detecteze, să decidă și să acționeze singur, lăsând controlul final la dispoziția administratorului.

## 1.3 Obiective

Obiectivul principal al lucrării este proiectarea și implementarea unui sistem de securitate adaptiv pentru servere Linux, capabil să transforme autonom detectarea unui atac într-o reacție concretă, acționând direct la nivelul sistemului gazdă.

În sprijinul acestui obiectiv, abordarea propusă se conturează în jurul a patru direcții cheie:

- Agregarea mai multor tipuri de semnale și evaluarea unitară a gradului de pericol reprezentat de o sursă, în locul analizării lor individuale.

- Menținerea unei reacții treptate și a controlului uman asupra deciziilor, astfel încât automatizarea să vină în ajutorul administratorului, nu să îl înlocuiască complet.
- Producerea unui efect real asupra sistemului protejat, utilizând instrumentele native ale gazdei.
- Asigurarea unei vizibilități complete asupra propriilor decizii, pentru ca sistemul să poată fi supravegheat și corectat la nevoie.

## 1.4 Soluția propusă

Soluția propusă este reprezentată de un sistem care verifică permanent jurnalele de autentificare și de nucleu, precum și conexiunile active de pe gazdă, transformând aceste informații în decizii de securitate. Periodic, sistemul adună semnalele suspecte, le unifică într-un scor unic per adresă și aplică automat reguli de firewall pentru acelea care depășesc pragurile stabilite în prealabil de administrator. Diferența fundamentală față de un filtru clasic constă în flexibilitatea sa, reacția nefiind fixă, ci dictată de politica de securitate setată, care poate fi modificată.

Construită în jurul acestui nucleu adaptiv, soluția îi garantează administratorului vizibilitate completă și control absolut. Fiecare avertisment sau acțiune de blocare automată generează o alertă afișată în timp real în interfața web, iar toate deciziile luate sunt stocate într-un istoric. Administratorul poate bloca sau debloca manual adrese IP, poate gestiona o listă de adrese de încredere și poate consulta statistici obținute pe baza istoricului. Mai mult decât atât, arhitectura integrează un asistent inteligent bazat pe un model de limbaj (LLM - Large Language Model), capabil să transforme cereri formulate în limbaj natural în acțiuni de firewall supuse confirmării umane, dar și un mecanism de acces securizat prin port knocking, care expune accesul la conectarea prin SSH doar la cerere.

## 1.5 Rezultatele obținute

În cadrul lucrării a fost realizat un sistem funcțional, capabil să detecteze automat sursele cu comportament suspect și să le blocheze la nivelul firewall-ului gazdei, însoțit de o interfață web care oferă o privire de ansamblu asupra stării serverului. Funcționarea corectă a aplicației a fost confirmată prin testarea completă a procesului de detecție și reacție. Evaluarea performanței a arătat că soluția operează cu un consum redus de resurse și cu latențe scăzute, ceea ce o face potrivită pentru utilizarea pe servere cu resurse limitate și o poziționează între soluțiile existente. Aceasta oferă mai multă vizibilitate și control față de instrumentele de protecție de bază, fără a fi necesară infrastructura complexă și consumul ridicat de resurse pe care le-ar presupune o platformă de tip SIEM (Security Information and Event Management).

## 1.6 Structura lucrării

Lucrarea este structurată în şapte capitole:

- *Introducere* - Prezintă contextul, problema, obiectivele, soluția propusă și rezultatele obținute, oferind o privire de ansamblu asupra lucrării.
- *Analiza cerințelor* - Pornind de la utilizatorul vizat și de la scenariile de utilizare, sunt definite funcționalitățile pe care aplicația trebuie să le ofere.
- *Studiu de piață* - Sunt analizate soluțiile existente pe piață, limitările acestora și oportunitatea oferită, alături de tehnologiile alese pentru implementare.
- *Soluția propusă* - Este descrisă arhitectura sistemului și modul în care componentele interacționează.
- *Detalii de implementare* - Descrie organizarea codului și implementarea efectivă a componentelor.
- *Evaluare* - Descrie testarea sistemului și măsoară performanța prin timpii de răspuns și consumul de resurse.
- *Concluzii* - Prezintă concluziile lucrării și direcțiile de dezvoltare ulterioară.

## 2 ANALIZA CERINȚELOR

### 2.1 Utilizatorul vizat

Produsul dezvoltat este destinat administratorilor de sistem, care întrețin unul sau câteva servere Linux expuse în internet, fiecare protejat individual. De exemplu, o organizație mare dispune de o echipă mare de securitate și de un buget pentru soluții premium de pe piață, dar un utilizator simplu nu are asemenea resurse și nici timpul necesar pentru a supraveghea permanent activitatea de pe sistem. Acest produs vine astfel în sprijinul său, oferind automatizarea necesară, dar nefiind nevoie ca acesta să cedeze controlul total unei aplicații.

O soluție utilă pentru un astfel de utilizator trebuie să acopere mai multe nevoi deodată. Una dintre nevoi este detectarea și oprirea atacurilor fără a interveni manual, deoarece utilizatorul nu poate supraveghea nonstop asupra sistemului. Vizibilitatea și posibilitatea de a vedea rapid ce se întâmplă pe server, ce adrese au fost blocate și de ce, fără a parcurge manual fișiere, dar și puterea de a ajusta comportamentul sistemului și corectarea eventualelor acțiuni greșite, reprezintă alte două nevoi ce trebuie acoperite.

Nevoile enumerate mai sus, dar și analiza soluțiilor existente pe piață definesc punctul de reper pentru cerințele produsului dezvoltat. Profilul utilizatorului descris anterior joacă rolul unei persona pe baza căreia au fost formulate scenariile de utilizare și funcționalitățile.

### 2.2 Scenarii de utilizare

Figura 15 prezintă cum a fost modelată interacțiunea cu sistemul sub forma unor cazuri de utilizare. Au fost identificați trei actori, și anume utilizatorul implicit, administratorul și planificatorul automat. Primii doi reprezintă partea umană, cu drepturi diferite, iar al treilea reprezintă componenta care rulează continuu, în cicluri, fără intervenția unui utilizator uman.

Utilizatorul implicit are drepturi de vizualizare. El poate consulta panoul central (dashboard), poate urmări conexiuni active și reguli firewall de pe gazdă și poate vedea statistici, grafice din pagina de analiză. Un administrator are aceste drepturi, dar, în plus, are dreptul de a executa acțiuni privilegiate, cum ar fi blocarea și deblocarea manuală a adreselor, controlarea listei de adrese de încredere, consultarea auditului, și cel mai important, poate modifica politica de securitate curentă.

Două dintre scenariile de utilizare ale administratorului includ obligatoriu un alt caz. Blocarea manuală a unei adrese IP include închiderea conexiunilor active ale acesteia, pentru ca măsura

să aibă efect imediat, iar formularea unei cereri în limbaj natural are nevoie de confirmarea acțiunii propuse, asistentul neputând să execute nimic singur, doar sugerează o acțiune pe care administratorul trebuie să o valideze înainte de a fi aplicată.

Planificatorul automat declanșează analizarea și detectarea automată a atacurilor din care se pot extinde, două comportamente, în funcție de pragul depășit. Blocarea automată include, de asemenea, închiderea conexiunilor și notificarea în interfață a acțiunii abia executate, iar avertizarea include doar notificarea. Acțiunile inițiate de om rămân separate față de cele declanșate automat, deși ambele produc, în final, aceleași efecte pe sistemul gazdă.

## 2.3 Cerințe funcționale

Din profilul utilizatorului și din cazurile de utilizare rezultă cerințele funcționale, adică capacitățile concrete pe care produsul trebuie să le ofere. Pentru claritate, acestea sunt grupate pe arii funcționale.

### *Detectie și evaluare*

- Identificare amenințări multiple - Sistemul trebuie să recunoască automat principalele tipare de atac la care este supus un server expus în internet, cum ar fi forța brută SSH (SSH brute force), enumerarea utilizatorilor (SSH user enumeration), scanarea de porturi (port scanning) sau inundarea cu conexiuni (connection flood).
- Calculare a unui scor unificat per adresă - Evenimentele provenite din surse diferite trebuie însumate pentru fiecare adresă, reflectând nivelul total de pericol al sursei respective, în locul tratării acestora izolat.

### *Răspuns treptat*

- Avertizare - La depășirea pragului de avertizare, adresa trebuie marcată ca fiind suspectă, dar fără a fi încă blocată
- Blocare - Administratorul trebuie să poată activa sau dezactiva blocarea automată, iar la depășirea pragului adresa trebuie blocată și conexiunile active deja stabilite ale acelei surse trebuie întrerupte pentru ca acțiunea să producă efect pe loc.

### *Configurare în timpul rulării*

- Politică editabilă - Pragurile, fereastra de analiză, dar și detectoarele active trebuie să poată fi configurate în timpul funcționării, fără a reporni întregul sistem, modificările având efect aproape imediat, în următorul ciclu.

### *Control manual*

- Blocare și deblocare manuală - Administratorul trebuie să poată bloca sau debloca o adresă prin aplicarea regulilor firewall în orice moment, independent de deciziile automate luate în sistem.
- Listă de adrese de încredere - Adresele trebuie să poată fi marcate ca fiind de încredere, acestea fiind excluse de la orice blocare automată.

### *Vizibilitate și audit*

- Dashboard și alerte - Starea curentă trebuie afișată într-un panou central, iar avertizările și blocările automate trebuie notificate în timp real.
- Istoric - Fiecare acțiune trebuie să fie auditată împreună cu motivul, autorul ei și momentul producerii, pentru a putea fi consultată ulterior.
- Analiză și statistici - Sistemul trebuie să ofere statistici pe o anumită perioadă, precum numărul de acțiuni executate pe fiecare categorie sau cele mai active adrese cu comportament suspect.
- Monitorizare a gazdei - Conexiunile active și regulile de firewall curente de pe gazdă trebuie să fie afișate în interfață.

### *Acces și securitate*

- Acces pe roluri - Acțiunile sensibile trebuie să fie permise numai utilizatorilor cu rolul de administrator, celălalt tip, utilizator implicit, având doar drept de consultare.
- Acces securizat la SSH - Sistemul trebuie să ofere un mecanism de a deschide portul SSH doar la cerere, ținându-l închis în restul timpului, prin port knocking .

### *Asistență cu inteligență artificială*

- Interpretare cereri - Sistemul trebuie să poată transforma o cerere formulată în limbaj natural într-o acțiune efectivă, care trebuie validată de către utilizator înainte de a fi executată.

## **2.4 Cerințe non-funcționale**

În plus față de funcționalitățile extrase din cazuri de utilizare specifice, produsul trebuie să îndeplinească și o serie de cerințe de calitate, care privesc modul în care sistemul îndeplinește sarcinile pentru care a fost creat.

- **Performanță** - Reacția sistemului este aproape imediată, între depășirea pragului de blocare până la aplicarea acțiunii trecând doar câteva secunde. Consumul de resurse

este redus deoarece supravegherea este realizată la o anumită perioadă, nu continuu, iar notificările apar în interfață aproape instantaneu la producerea evenimentului, fiind transmise prin WebSocket, fără reîncărcarea paginii.

- **Securitate** - Autentificarea în aplicație este implementată folosind o soluție deja consacrată, iar verificarea drepturilor se face pe server, pe fiecare punct de acces (endpoint) disponibil, nu doar prin ascunderea paginilor din interfața web. Datele de securitate rămân pe mașina locală, mai puțin cererile scrise în limbaj natural care sunt trimise către API-ul extern al modelului lingvistic (LLM).
- **Fiabilitate** - Regulile de firewall sunt aplicate la nivelul gazdei, deci rămân salvate indiferent de starea aplicației, iar la pornire sistemul își reconstruiește starea de lucru din baza de date, reîncărcând adresele urmărite și statusul acestora împreună cu valorile de referință, pentru a nu bloca din greșeală o adresă deblocată anterior.
- **Mentenabilitate** - Sistemul poate fi ușor extins cu noi tipuri de detectoare fără a modifica componentele existente.
- **Utilizabilitate** - Întreaga interacțiune se desfășoară printr-o interfață web, care înlocuiește editarea manuală a configurărilor și rularea comenzilor din linia de comandă, iar administratorul poate face de aici acțiunile necesare, fără a interveni direct pe gazdă.

## 2.5 Cerințe de sistem

Pentru a funcționa în parametri normali, soluția are nevoie de următoarele cerințe de sistem:

- Sistem de operare: Linux
- Mediu de rulare: JDK (Java Development Kit) 21
- Bază de date: PostgreSQL 18
- Furnizor de identitate: Keycloak 26.0.0
- Containerizare: Docker și Docker Compose
- Browser: Google Chrome, Mozilla Firefox

## 2.6 Constrângeri

În continuare sunt explicate limitele și regulile de bază ale sistemului, în ce situații funcționează soluția și ce nu își propune să rezolve la acest moment.

- O singură gazdă - Sistemul protejează un singur server, pe care rulează, coordonarea mai multor mașini dintr-un punct central nefăcând parte din obiectivele curente.
- Mediu Linux - Soluția necesită un sistem de operare Linux, bazându-se pe utilitățile acestuia. Pentru a funcționa pe un alt sistem de operare, aceste componente ar trebui ajustate.

- Privilegii - Aplicația are nevoie de permisiunile necesare pentru a rula anumite comenzi la nivelul sistemului de operare.
- Atacuri vizibile - Soluția detectează atacuri de la nivelul gazdei, dar acțiunile la nivelul aplicației, precum injecțiile SQL (Structured Query Language) sau XSS (Cross-Site Scripting) nu intră în scop.
- Atacuri individuale - Aplicația protejează împotriva surselor individuale. Dacă atacul este distribuit, mai multe surse atacă minor sistemul, aplicația le ignoră, întrucât acestea nu depășesc pragurile individual.



### 3 STUDIU DE PIAȚĂ

Problema protejării automate a serverelor Linux nu este nouă, iar pe piață există mai multe soluții, utilitare simple și gratuite sau platforme profesionale complexe. Această secțiune analizează cele mai relevante opțiuni disponibile și evidențiază neajunsurile în raport cu profilul utilizatorului țintă și oportunitatea rezultată.

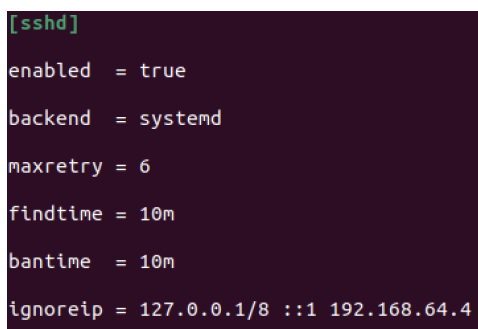
Pentru ca evaluarea să fie una relevantă, soluțiile sunt analizate pe baza criteriilor specifice profilului de utilizator descris anterior. Automatizarea, transparența, complexitatea și securitatea datelor sunt punctele de reper pentru această evaluare. Comparăția făcută arată exact ce nevoi rămân neacoperite și justifică direcția aleasă pentru noul sistem.

#### 3.1 Soluții existente

##### 3.1.1 Fail2ban

Unul dintre cele mai utilizate instrumente din această categorie este *fail2ban*, care monitorizează jurnalele serviciilor și blochează prin firewall adresele ale căror evenimente depășesc un anumit prag prestabilit într-un interval de timp [3]. La baza configurării stau închisorile (jails), fiecare dintre acestea combinând o sursă de jurnal, un filtru și o acțiune, împreună cu parametrii care stabilesc regulile de blocare (numărul maxim de încercări, adresele IP care trebuie ignorate, fereastra de timp, durata blocării, etc.). Atunci când o sursă depășește numărul permis de încercări în fereastra de observare, utilitarul adaugă automat o regulă de firewall care o blochează pe durata configurată. Soluția are în mod implicit filtre predefinite pentru numeroase servicii, precum SSH, servere web, servere de mail sau servere FTP (File Transfer Protocol), dar poate fi și extins prin adăugarea de filtre proprii.

Mai jos este prezentat un astfel de fișier de configurare (jail.local) pentru serviciul SSH.



```
[sshd]
enabled = true
backend = systemd
maxretry = 6
findtime = 10m
bantime = 10m
ignoreip = 127.0.0.1/8 ::1 192.168.64.4
```

Figura 1: Fișier de configurare fail2ban pentru serviciul SSH

Ușurința de utilizare, consumul redus de resurse, vechimea, stabilitatea, și răspândirea la scară largă sunt punctele sale forte prin care acesta se remarcă. Este inclus în depozitele oficiale ale principalelor distribuții Linux și este eficient împotriva atacurilor de tip brute force obișnuite. Totuși, această simplitate arată unde soluția nu mai face față nevoilor utilizatorului vizat. Fiecare jail funcționează într-un mod izolat, analizând evenimentele independent de celelalte, fără un scor agregat din mai multe servicii diferite. Neexistând nicio interfață web și niciun istoric vizual, configurarea și consultarea se fac prin fișiere și linia de comandă. De asemenea, o blocare actualizează regulile de firewall pentru a respinge doar conexiunile noi [3], dar sesiunile deja active înainte de asta rămân active în mod implicit.

```
ubuntu@ubuntu:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 1
|   |- Total failed: 31
|   `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
|- Actions
|   |- Currently banned: 1
|   |- Total banned: 1
|   `-- Banned IP list: 192.168.64.1
ubuntu@ubuntu:~$
```

Figura 2: Starea jail-ului SSH

### 3.1.2 CrowdSec

O altă soluție existentă pe piață este *CrowdSec*, care păstrează detecția pe baza jurnalelor, dar este construită pe o altfel de arhitectură (Figura 3). La fel ca fail2ban, aceasta pornește de la citirea jurnalelor de sistem pentru a detecta activitățile suspecte, însă modul în care este construită în spate diferă foarte mult, atât prin separarea componentelor cât și prin comunicarea cu alte servere pentru a bloca atacatorii.

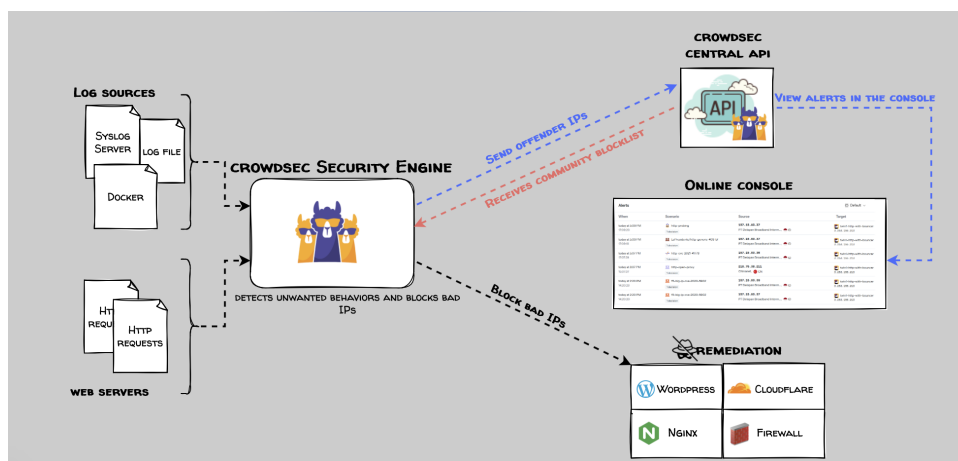


Figura 3: Arhitectura CrowdSec<sup>1</sup>

<sup>1</sup><https://docs.crowdsec.net/docs/intro/>

În aplicațiile mai vechi, o singură componentă realizează tot procesul, detecția și blocarea, dar CrowdSec separă cele două operațiuni, delegându-le unor module ce comunică între ele printr-un API local (LAPI). Un agent, care funcționează ca un analizator, citește în permanență jurnalele, prelucrează informația și evaluează pe baza unor scenarii de atac pentru a găsi adresele IP periculoase. Acesta nu aplică nicio blocare, ci înregistrează deciziile și le pune la dispoziția celorlalte componente, iar de aplicarea efectivă sunt responsabile niște componente numite *bouncers*, care preiau deciziile și interzic accesul adreselor suspecte. Decuplarea în componente ajută la flexibilitate pentru că decizia este independentă de modul de aplicare, iar restricția nu trebuie impusă neapărat printr-o regulă de firewall. Un *bouncer* poate rula direct pe serverul web, afișându-i atacatorului o pagină de eroare, sau chiar la nivelul unei rețele de distribuție (CDN - Content Distribution Network), oprind traficul periculos înainte ca acesta să ajungă la server.

Un avantaj major al soluției CrowdSec este că adună utilizatorii într-o rețea globală pentru a colabora, nefiind gândit să funcționeze izolat. Agentul poate să se conecteze la un serviciu central cu care face schimb în ambele sensuri. În momentul în care este atacat, el trimite către API-ul central (CAPI) adresa de la care vine atacul împreună cu tipul de comportament observat, iar, în același timp, descarcă liste cu adrese periculoase, semnalate de celelalte servere din această rețea. Înainte de a ajunge în listele partajate cu ceilalți utilizatori, adresele semnalate sunt verificate central pentru a reduce erorile. Abordarea oferă un mecanism de protecție în avans, prin care un atacator detectat oriunde în lume este blocat automat pe serverul propriu încă dinainte ca acesta să încerce un nou atac.

Faptul că este construit pe o arhitectură decuplată face ca soluția să fie scalabilă, potrivită mai ales organizațiilor cu infrastructuri mari, formate din numeroase servere. De asemenea, limbajul de programare în care este scrisă, Go, oferă un consum redus de resurse, procesând eficient seturi mari de evenimente. Capacitatea de colaborare are la bază trimiterea informațiilor despre atacuri către un serviciu central și pe consultarea unor liste din exterior, ceea ce înseamnă o dependență față de o componentă din afara sistemului și partajarea datelor către exterior, un compromis pe care un utilizator ce dorește o soluție cu date păstrate local ar evita să îl facă. Mai mult, administratorul trebuie să instaleze și să configureze separat agentul și *bouncer*-ele și să se asigure că API-ul local permite comunicarea [5], ceea ce înseamnă complexitate și efort în plus pentru un utilizator care administrează un singur server, nevoia fiind una de o aplicație compactă, gata de folosit.

### 3.1.3 Wazuh

*Wazuh* este o soluție de nivel mai înalt și este o platformă de tip SIEM. Aceasta nu se limitează la blocarea adreselor cu comportament abuziv, ci urmărește întreaga activitate de securitate a unui sistem. Niște agenți instalați pe fiecare gazdă monitorizată colectează informații din jurnale, evenimente și date despre stare și le trimit către un server central. Datele sunt decodificate de către server, trecute printr-un set de reguli și transformate în alerte, iar mai

apoi sunt salvate într-o bază de date și afișate într-un dashboard. Pe lângă citirea jurnalelor, Wazuh oferă și funcții precum monitorizarea integrității fișierelor sau detecția vulnerabilităților, dând o perspectivă de ansamblu asupra securității unui server. Arhitectura sa este ilustrată în Figura 4.

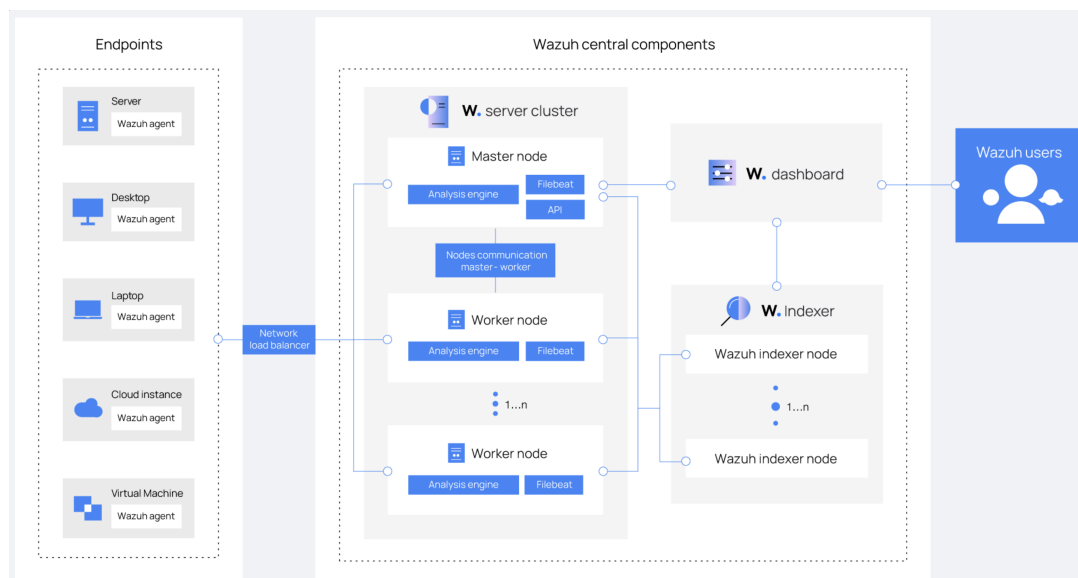


Figura 4: Arhitectura Wazuh<sup>2</sup>

Adunând la un loc evenimentele de pe mai multe gazde, sistemul reușește să le pună într-un context general și prezintă o imagine de ansamblu pe care uneltele punctuale, precum fail2ban, nu au cum să o ofere. Este o soluție cu vechime și folosită cu succes de echipe de securitate pentru monitorizarea incidentelor la scară largă.

Amploarea soluției, însă, o face nu tocmai potrivită pentru cazul de față. Wazuh este alcătuit din mai multe componente, iar instalate pe aceeași gazdă, acestea au nevoie de resurse considerabile [9]. De asemenea, valoarea ei reală apare atunci când există o echipă dedicată care să interpreteze alertele și să ajusteze regulile, blocarea automată fiind posibilă ca un mecanism secundar, dar nu scopul principal al aplicației.

### 3.1.4 Alte soluții

Pe piața de soluții de securitate se află și alte soluții, situate însă la alt nivel sau care au alt scop. Sistemele de detecție și prevenție a intruziunilor (IDS/IPS), precum Snort sau Suricata [19], inspectează pachetele în funcție de anumite semnături și își focalizează activitatea pe trafic, nu pe jurnalele de pe gazdă. Firewall-urile pentru aplicații web, precum ModSecurity [20], protejează stratul HTTP (Hypertext Transfer Protocol) printr-un mecanism ce oferă un punctaj anomaliilor [2], apropiat, ca idee, de scorul pentru fiecare adresă folosit în această lucrare, dar aplicat cererilor web, nu accesului la serviciile gazdei.

<sup>2</sup><https://documentation.wazuh.com/current/getting-started/architecture.html>

Tabela 1 arată o comparație a soluțiilor analizate mai sus, raportată la cerințele proiectului.

Tabela 1: Comparație soluții existente

| Aplicație              | Automatizare                   | Vizibilitate și control         | Complexitate | Date locale                |
|------------------------|--------------------------------|---------------------------------|--------------|----------------------------|
| <b>fail2ban</b>        | Da (per serviciu)              | Minimă (fișiere și CLI)         | Redusă       | Da                         |
| <b>CrowdSec</b>        | Da (comportament)              | Bună (consolă web)              | Medie        | Parțial (serviciu central) |
| <b>Wazuh</b>           | Da (funcționalitate secundară) | Foarte bună (dashboard complet) | Ridicată     | Da                         |
| <b>Soluția propusă</b> | Da (evenimente multiple)       | Minima (fișiere și CLI)         | Redusă       | Da                         |

## 3.2 Oportunitatea

Pe baza acestei analize se evidențiază un spațiu neacoperit, situat între instrumentele prea simple, de tip fail2ban, fără scor agregat și cu vizibilitate redusă, și soluțiile prea complexe sau dependente de exterior, precum Wazuh, platformă de tip SIEM gândită pentru organizații cu echipe dedicate pentru securitate. Astfel, lucrarea de față se plasează în acest spațiu, propunând o soluție de sine stătătoare, destinată administratorului unui singur server, care reunește mai multe tipuri de evenimente, reacționează automat și păstrează datele și deciziile luate local, oferind vizibilitate completă printr-o interfață web.

## 3.3 Tehnologiile utilizate

Soluția implementată folosește un set de tehnologii alese în urma comparării cu alternativele lor. Acestea sunt prezentate în Tabela 2.

Tabela 2: Tehnologii utilizate

| Tehnologie                    | Alternative                  | Motiv                                          |
|-------------------------------|------------------------------|------------------------------------------------|
| <b>Java &amp; Spring Boot</b> | Node.js, Django              | Ecosistem stabil, care are tot ce este necesar |
| <b>Angular</b>                | React, Vue                   | Cadru (framework) complet și structurat        |
| <b>PostgreSQL</b>             | MySQL, MongoDB               | Bază de date relațională, fiabilă              |
| <b>Keycloak</b>               | Auth0, JWT propriu           | Open-source, self-hosted                       |
| <b>Docker</b>                 | Podman, rulare nativă        | Izolare și reproductibilitate                  |
| <b>iptables</b>               | nftables                     | Universal, control direct                      |
| <b>Google Gemini</b>          | OpenAI GPT, Anthropic Claude | Gratuit, rezultate structurate                 |

### 3.3.1 Backend

Pentru partea de server a fost ales limbajul de programare Java, împreună cu framework-ul Spring Boot, în locul alternativelor (Node.js sau Django). Node.js are un ecosistem vast, însă obligă dezvoltatorul să configureze manual elemente de bază, precum securitatea sau accesul la date. De asemenea, atât Node.js cât și Django au tipare dinamică, ceea ce scade gradul de siguranță al codului. Spring Boot oferă nativ aceste componente într-un ecosistem unitar, integrând securitatea OAuth2, planificarea sarcinilor, comunicarea prin WebSocket și accesul la baza de date (Spring Data JPA) [8].

### 3.3.2 Frontend

React este, în principiu, o bibliotecă de interfață (UI) care necesită configurarea manuală a multor piese (rutare, formulare) [7]. La rândul său, Vue este un framework flexibil, acoperind în principal interfața, dar rutarea sau injectia de dependențe vin ca biblioteci separate și organizarea aplicației rămâne la latitudinea dezvoltatorului. Angular a fost ales fiindcă oferă din start toate uneltele necesare, nu trebuie instalate componente externe pentru funcționalități de bază, pentru că le are deja incluse, iar structura ordonată a codului pe care o impune reprezintă un avantaj în dezvoltarea aplicației [1].

### 3.3.3 Bază de date

Pentru stocarea informațiilor a fost ales PostgreSQL deoarece este o soluție fiabilă care se potrivește cu datele structurate ale aplicației. MySQL este tot o bază de date relațională populară și rapidă, la fel de potrivită datelor aplicației, astfel că alegerea dintre aceasta și PostgreSQL este una de preferință, PostgreSQL fiind recunoscut pentru fiabilitate și robustețe [6] și cea mai folosită bază de date conform sondajului Stack Overflow [23]. MongoDB, însă, este o bază de date NoSQL, orientată pe documente [17], care s-ar potrivi cu date nestructurate, însă datele aplicației nu se aliniază cu această soluție deoarece ele au o schemă fixă și clar definită.

### 3.3.4 Autentificare și identitate

Auth0 este o soluție de identitate ca serviciu (IDaaS), ușor de integrat, dar este dependent de un furnizor. De altfel, o implementare proprie ar fi însemnat rescrierea celei mai sensibile părți ale aplicației, cu riscuri ridicate de securitate. Astfel, a fost ales Keycloak, deoarece este un furnizor de identitate open-source și complet gratuit, care oferă OAuth2 și control pe roluri, fără cost și cu datele ținute local.

### **3.3.5 Containerizare**

Serviciile auxiliare, baza de date și cel de autentificare, rulează în containere Docker. Instalarea acestora direct pe sistemul de operare nu a fost o opțiune de luat în calcul pentru că ar fi presupus mai mult efort manual. Printre instrumentele de containerizare a fost evaluat și Podman, similar cu tehnologia aleasă, dar Docker a rămas decizia finală datorită stabilității și documentației vaste. Docker izolează perfect și este portabil, ridicând ambele servicii cu versiuni fixe, păstrând sistemul curat, printr-un singur fișier de configurare.

### **3.3.6 Aplicare**

Pentru aplicarea regulilor de firewall, alternativa principală a iptables a fost nftables, succesorul modern al acestuia [18]. Utilitarul iptables a fost ales pentru disponibilitatea universală pe orice server și pentru controlul direct asupra regulilor, necesar mecanismelor de blocare și port knocking.

### **3.3.7 Asistență**

Atât Google Gemini, cât și OpenAI GPT sau Anthropic Claude, sunt modele comerciale ce sunt capabile să ofere ieșirea structurată de care aplicația are nevoie, însă, cele două alternative necesită costuri de utilizare. Astfel, Google Gemini a fost ales pentru că acoperă cerința și are un nivel gratuit suficient.

## 4 SOLUȚIA PROPUȘĂ

Soluția propusă în această lucrare este un sistem adaptiv de politici de securitate care monitorizează permanent un server Linux. Aplicația evaluează și identifică în mod automat adresele IP cu comportament suspect și, la nevoie, le blochează. Totodată, sistemul oferă administratorului control deplin și vizibilitate asupra acțiunilor efectuate prin intermediul unei interfețe web structurate sugestiv. Spre deosebire de un firewall clasic, în care regulile de securitate sunt configurate manual și rămân fixe, soluția propusă își ajustează comportamentul în funcție de activitatea observată și de politicile stabilite de către administrator, care pot fi schimbate în orice moment fără a fi necesară repornirea aplicației.

Acest capitol prezintă arhitectura sistemului din două perspective diferite:

- Structura componentelor - părțile componente și cum comunică între ele
- Fluxul aplicației - modul în care informația circulă, de la observarea unui atac la blocarea automată și notificarea administratorului

### 4.1 Vedere în ansamblu asupra arhitecturii

La nivel conceptual, sistemul funcționează ca un paznic atent, un mecanism de monitorizare continuă ce urmărește încercările de acces la server, recunoaște tiparele de comportament suspect și, pe baza regulilor clar stabilite, decide când o entitate externă trebuie blocată, toate aceste acțiuni fiind auditate.

La nivel tehnic, structura internă a sistemului este organizată pe trei niveluri, interconectate cu o serie de servicii externe specializate. Această separare urmează un principiu clasic de proiectare, fiecare strat având o singură responsabilitate, ceea ce face ca sistemul să fie ușor de înțeles, de testat și de extins. Structura generală este ilustrată în Figura 5.



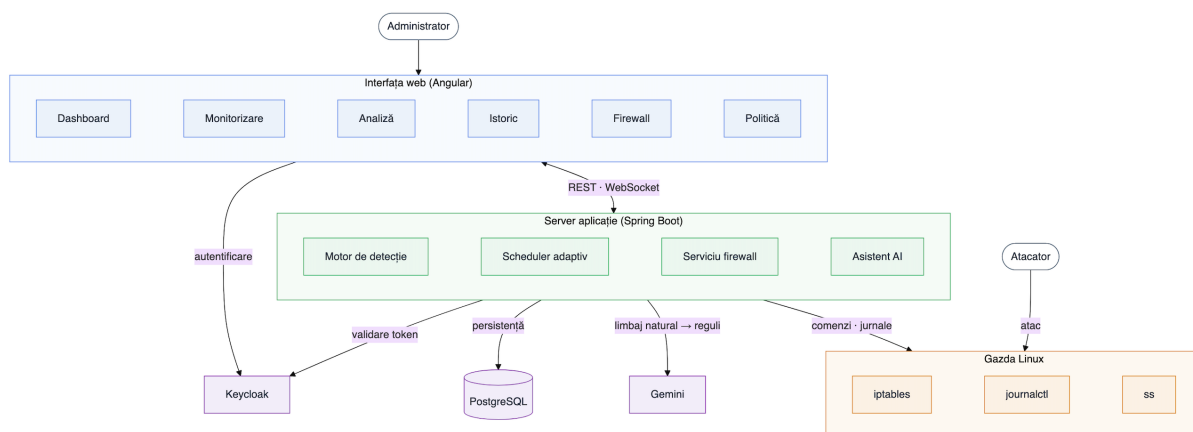


Figura 5: Arhitectura generală a sistemului

### 4.1.1 Nivelul de prezentare (interfața web)

Acesta reprezintă singurul strat arhitectural cu care utilizatorul interacționează în mod direct. A fost implementat sub forma unei aplicații web, aceasta fiind organizată în pagini dedicate:

- panou de vizualizare general (dashboard)
- secțiune pentru monitorizarea conexiunilor și a regulilor firewall
- secțiune de analiză cu statistici
- istoricul acțiunilor din sistem
- secțiune pentru administrarea regulilor firewall - blocare/deblocare și asistență printr-un modul de inteligență artificială
- pagină de configurare a politicilor de securitate și lista de adrese IP de încredere

### 4.1.2 Nivelul de logică (serverul aplicației)

Acest nivel reprezintă nucleul decizional al sistemului. Printre responsabilitățile sale se numără analizarea periodică a jurnalelor de sistem și a conexiunilor active, agregarea semnalelor de atac, luarea deciziilor de avertizare sau blocare conform politicii de securitate setată. De asemenea, acest nivel expune o interfață de programare (API) necesară comunicării cu interfața web și gestionează transmiterea notificărilor în timp real. În esență, această componentă face legătura între interfața cu care interacționează utilizatorul și operațiile efective executate la nivelul sistemului de operare.

### 4.1.3 Nivelul de execuție (gazda Linux)

Acesta este stratul în care deciziile de securitate sunt efectiv aplicate. Serverul aplicației rulează comenzi specifice sistemului de operare pentru a gestiona regulile firewall, a inspecta conexiunile de rețea active sau a analiza jurnalele de autentificare.

#### 4.1.4 Servicii externe

Cele trei niveluri de bază sunt susținute de o suită de servicii externe. Persistența datelor este asigurată de o bază de date centralizată, care stochează adresele monitorizate, istoricul blocărilor și al tuturor acțiunilor efectuate, lista de adrese de încredere și politicile de securitate. Accesul la aplicație este protejat printr-un serviciu de identitate care gestionează autentificarea utilizatorilor și controlul accesului bazat pe roluri (RBAC - Role-Based Access Control) [22], astfel încât acțiunile privilegiate sunt permise numai persoanelor autorizate, și anume administratorilor. De asemenea, sistemul integrează un LLM care stă la baza unui asistent inteligent. Acesta interpretează o cerere descrisă în limbaj natural și propune adăugarea de reguli firewall pe baza acesteia. Acțiunile propuse trebuie validate de către utilizator pentru a fi executate.

#### 4.1.5 Acces securizat prin port knocking

Pe lângă mecanismele de combatere a amenințărilor, aplicația încorporează funcția de *port knocking* [13] pentru a permite accesul de la distanță fără a lăsa portul expus permanent. Portul SSH este implicit închis și se deschide dinamic doar pentru adresele IP care trimit o secvență predefinită de cereri de conectare, într-o ordine exactă și într-un interval de timp setat. După o secvență validă, portul rămâne deschis pentru acea adresă pentru o fereastră de timp limitată, urmând să fie închis automat. În acest fel, serviciul de administrare devine practic invizibil pentru un atacator, dar disponibil, la cerere, pentru persoane autorizate.

### 4.2 Fluxul adaptiv

Caracterul adaptiv al sistemului este dat de un ciclu automat care se repetă la un scurt interval de timp și transformă jurnalele sistemului de operare în decizii pentru securitatea gazdei. Acest ciclu ține cont în permanență de politicile de securitate configurate de către administrator, care pot fi ajustate în orice moment. Etapele ciclului sunt ilustrate în Figura 6.

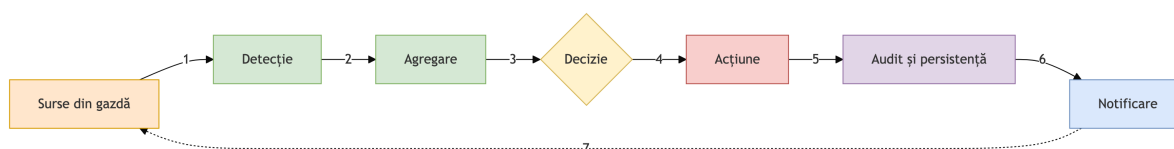


Figura 6: Fluxul adaptiv

#### 1. Colectare semnale

Sistemul citește jurnalele de autentificare și de nucleu de pe mașina gazdă și inspectează conexiunile active. Acesta urmărește adresele IP care au un comportament suspect, pre-

cum multiple încercări eșuate de autentificare SSH, încercări de conectare cu utilizatori inexistenți, scanări pe mai multe porturi sau un număr mare de conexiuni simultane.

## **2. Decizie**

Pentru identificarea potențialelor atacuri, platforma folosește mai multe detectoare independente, fiecare specializat pe un anumit tip de atac. Acestea identifică adresele IP implicate și numără evenimentele înregistrate pentru categoria corespunzătoare. Acest proces este configurabil, administratorul având posibilitatea de a activa sau dezactiva fiecare detector în parte.

## **3. Agregare**

Evenimentele provenite de la fiecare detector sunt însumate pentru fiecare adresă. Această sumă reflectă cât de intens este atacată gazda de către acea adresă.

## **4. Decizia**

Scorul calculat în etapa anterioară este comparat cu pragurile definite în politica stabilită de administrator, unul de avertizare și unul de blocare. La acest pas, sistemul ia în considerare adresele din lista de încredere, care nu vor fi blocate niciodată, și un mecanism care previne reblocarea unei adrese deblocate manual din aplicație.

## **5. Acțiune**

În cazul în care scorul trece de pragul de avertizare, adresa este marcată ca suspectă. Dacă depășește și pragul de blocare, sistemul introduce în mod automat o regulă firewall care blochează accesul și închide toate conexiunile active, astfel încât blocarea să aibă efect imediat.

## **6. Persistență și audit**

Fiecare avertizare și blocare a unei adrese sunt salvate în baza de date împreună cu motivul, autorul acțiunii și momentul auditării.

## **7. Notificare**

În cele din urmă, utilizatorul este înștiințat în timp real printr-o alertă afișată în interfața web, care conține adresa vizată, tipul evenimentului, motivul și numărul de semnale.

Elementul cheie care face sistemul adaptiv este faptul că atât pragurile, cât și detectoarele active fac parte din politica de securitate care este editabilă în timp ce aplicația rulează. Astfel, administratorul poate înăspri sau relaxa reacția sistemului, fără a întrerupe funcționarea sistemului.

## **4.3 Scenariu de utilizare**

Pentru a clarifica cum funcționează cele trei niveluri, urmăm un scenariu concret. Să presupunem că un atacator încearcă repetat să ghicească parola de acces la server.

La nivelul gazdei, fiecare încercare eșuată este înregistrată în jurnal. La următoarea rulare a ciclului de analiză, serverul aplicației observă aceste încercări, iar după depășirea pragului de avertizare, adresa este marcată ca suspectă. La depășirea pragului de blocare, sistemul

introduce automat regula firewall. În același moment, administratorul vede în dashboard alerta.

Din acel moment, administratorul are acces deplin. El poate consulta istoricul acțiunilor și poate debloca adresa sau o poate adăuga în lista adreselor de încredere. Acesta poate chiar să formuleze o cerere în limbaj natural, spre exemplu: "Blochează tot traficul care vine din subrețeaua x.x.x.x/y". Cererea este mai departe interpretată de modulul de inteligență artificială și tradusă într-o acțiune ce necesită validare manuală.

## 5 DETALII DE IMPLEMENTARE

Acest capitol descrie cum a fost de fapt implementat sistemul descris la nivel conceptual în capitolul precedent. Acoperă modul în care codul sursă este organizat și implementarea efec-tivă a fiecărei componente, precum motorul de detecție, autentificarea, scanarea conexiunilor, notificările în timp real, și asistentul alimentat de inteligență artificială.

### 5.1 Organizarea codului

Arhitectura backend-ului este organizată după responsabilitățile funcționale, respectând astfel principiul SoC (Separation of Concerns). Codul Java este împărțit în pachete clar delimitate, și anume:

- *controller* - gestionează interfața HTTP și rutarea cererilor
- *service* - cuprinde logica de business a aplicației
- *repository* - oferă un strat de abstractizare pentru accesul la baza de date
- *entity* - modelul de date pentru maparea obiect-relațională (ORM - Object Relational Mapper)
- *dto* - definirea obiectelor de transfer de date folosite în comunicarea cu interfața web
- *config* - grupează configurările sistemului

Această organizare păstrează fiecare strat simplu și specializat într-o anumită acțiune. Controller-ele nu realizează procesări complexe, ci se limitează la validarea cererilor, delegarea sarcinilor către serviciul potrivit și returnarea răspunsului. Logica de business se regăsește în întregime în servicii. Singura componentă care interacționează cu sistemul de operare de pe gazdă este un serviciu care execută comenzile, obținând astfel un punct unic de comunicare între aplicație și gazdă. Obiectele pentru transferul datelor (DTO) sunt intenționat separate de entitățile JPA (Java Persistence API), eliminând riscul ca modelul bazei de date să fie expus în specificațiile API-ului și cele două entități să evolueze independent. Frontend-ul se comportă în același fel, fiecare pagină fiind o componentă Angular de sine stătătoare, iar comunicarea cu backend-ul este izolată în servicii dedicate.

Decuplarea discutată nu este doar o convenție stilistică, aceasta oferind avantaje tehnice în ansamblul proiectului. Propagarea unei cereri care ajunge la serverul aplicației urmează un traseu bine definit. Cererea ajunge în sistem prin *controller*, care face o validare la suprafață, urmând ca aceasta să treacă mai departe către nivelul *service*, unde se execută logica aplicației. Nivelul *repository* gestionează persistența datelor, iar rezultatul final este transformat într-un

DTO înainte de a fi trimis ca răspuns. Astfel, impactul modificărilor făcute este izolat la nivelul componentei respective. În partea de backend, prin separarea claselor pentru baza de date și API, am putut restructura tabelele din bază fără a afecta compatibilitatea cu interfața web. De asemenea, în client, toate apelurile către server se fac prin servicii dedicate, astfel că orice schimbare a unei rute API se face într-un singur loc.

Clientul comunică cu serverul printr-un API de tip REST (Representational State Transfer), alături de un canal WebSocket destinat alertelor în timp real. API-ul este structurat pe resurse, iar fiecare endpoint este protejat. Toți utilizatorii au posibilitatea de a interoga date, pe când dreptul de a executa acțiuni critice sau urmărirea datelor sensibile revine numai administratorilor. Tabela 3 oferă o privire de ansamblu asupra rutelor.

Tabela 3: Rutele API-ului REST și nivelurile de acces

| Endpoint                                   | Acces         | Scop                                                           |
|--------------------------------------------|---------------|----------------------------------------------------------------|
| <b>GET</b> /api/monitoring/*               | Utilizator    | Conexiuni active, reguli firewall, adrese IP suspecte          |
| <b>GET</b> /api/analytics/overview         | Utilizator    | Statistici pentru grafice                                      |
| <b>GET</b> /api/policy                     | Utilizator    | Politica curentă                                               |
| <b>PUT</b> /api/policy                     | Administrator | Schimbare praguri și detectoare active                         |
| <b>POST</b> /api/firewall/block (/unblock) | Administrator | Blocare/deblocare manuală a adreselor                          |
| <b>GET</b> /api/whitelist                  | Utilizator    | Vizualizare listă de încredere                                 |
| <b>POST</b> /api/whitelist                 | Administrator | Adăugare adresă în lista de încredere                          |
| <b>DELETE</b> /api/whitelist/{id}          | Administrator | Ștergere adresă din lista de încredere                         |
| <b>GET</b> /api/events                     | Administrator | Vizualizare istoric acțiuni                                    |
| <b>POST</b> /api/ai/firewall/interpret     | Administrator | Transformare cerere din limbaj natural în propunere de acțiuni |

## 5.2 Motorul de detecție

Motorul de detecție este nucleul sistemului și a fost construit folosind o interfață pe care fiecare detector o implementează, astfel încât acestea pot fi tratate uniform, iar adăugarea de noi detectoare poate fi realizată fără a modifica restul aplicației.

Fiecare detector specifică categoria de atac pe care o acoperă, iar atunci când este interogată, returnează un dicționar cu adresele IP și numărul de evenimente pe care le-a observat în fereastra de timp specificată. Astfel, a fost aplicat șablonul de proiectare (design pattern) *Strategy* [14], fiecare algoritm de detecție fiind împachetat ca o strategie independentă, expusă printr-o interfață comună așa cum este ilustrat în Figura 7.

```

4
5 public interface ThreatDetector {
6     String category();
7     Map<String, Integer> detect(int windowMinutes);
8 }

```

Figura 7: Interfața *ThreatDetector*

Trei dintre cele patru detectoare citesc jurnalele sistemului și diferă doar prin comanda pe care o execută și tipul de acțiune vizată. Pentru a evita duplicarea codului, logica comună a fost extrasă într-o clasă abstractă de bază (Figura 8), care rulează comanda, tratează excepțiile și contorizează adresele IP identificate, lăsând la dispoziția claselor derivate furnizarea detaliilor de execuție suprascriind metoda *buildCommand*.

```

10 public abstract class LogPatternDetector implements ThreatDetector {
11     private final CommandExecutorService commandExecutor;
12     private final Pattern ipPattern;
13
14     > protected LogPatternDetector(CommandExecutorService commandExecutor, String ipRegex) {...}
15
16
17
18     protected abstract String buildCommand(int windowMinutes);
19
20
21     @Override
22     > public Map<String, Integer> detect(int windowMinutes) {...}
23
24
25
26
27
28
29 }
30

```

Figura 8: Clasa abstractă *LogPatternDetector*

Deoarece majoritatea logicii se regăsește în clasa de bază, un detector se reduce la câteva linii de cod. Detectorul de forță brută SSH (brute-force) citește jurnalul SSH și ține cont doar de liniile care semnalează o parolă greșită, în timp ce detectorul de probe SSH urmărește liniile care indică utilizatori inexistenți, ambele folosind același model de extragere a adresei IP.

Celelalte două module de detecție funcționează diferit. Cel destinat identificării scanărilor de porturi analizează jurnalele nucleului (kernel logs), unde o regulă de tip log din iptables înregistrează fiecare conexiune nouă folosind un prefix predefinit, iar sistemul contorizează porturile de destinație unice și marchează adresa IP ca fiind suspectă atunci când numărul de porturi interogate depășește pragul configurat în prealabil. Figura 9 prezintă metoda *detect* a acestui detector.

```

37     @Override
38     public Map<String, Integer> detect(int windowMinutes) {
39         Map<String, Integer> result = new HashMap<>();
40         String command = String.format(AppConstants.KERNEL_LOG_CMD, windowMinutes) + " | grep \"" + logPrefix + "\"";
41         String output = commandExecutor.execute(command);
42         if (output == null || output.isBlank())
43             || output.startsWith("Warning") || output.startsWith("Internal error")) {
44             return result;
45         }
46
47         Map<String, Set<String>> portsByIp = new HashMap<>();
48         for (String line : output.split(regex: "\n")) {
49             Matcher matcher = SCAN_PATTERN.matcher(line);
50             if (matcher.find()) {
51                 portsByIp.computeIfAbsent(matcher.group(1), k -> new HashSet<>()).add(matcher.group(2));
52             }
53         }
54
55         for (Map.Entry<String, Set<String>> entry : portsByIp.entrySet()) {
56             int distinctPorts = entry.getValue().size();
57             if (distinctPorts >= minPorts) {
58                 result.put(entry.getKey(), distinctPorts);
59             }
60         }
61         return result;
62     }
63 }

```

Figura 9: Metoda *detect* pentru scanare de porturi

Detectorul pentru inundațiile de conexiuni (connection flood) realizează o captură de stare a conexiunilor TCP (Transmission Control Protocol) stabilite la un moment dat folosind utilitarul *ss*. Acesta numără conexiunile simultane menținute active de fiecare adresă IP, iar cazurile care depășesc pragul setat sunt raportate.

Datorită funcției de injectare automată oferite de framework-ul Spring, toate instanțele (beans) care implementează interfața de detecție sunt adunate într-o listă, orchestratorul funcționând independent de numărul total de module de detecție existente, pentru fiecare în parte verificându-se dacă este activ în politica de securitate curentă. Dirijorul declanșează execuția pe fereastra de timp prestabilită și cumulează rapoartele individuale într-un singur scor total pentru fiecare adresă IP.

Scorul total per adresă IP este stocat într-un obiect a cărui metodă *addSignal* calculează suma înregistrărilor și reține fiecare categorie care a contribuit la aceasta. Acesta, care indică cât de suspectă este activitatea asociată adresei IP, urmează modelul de atribuire a unui scor de anomalie (anomaly scoring [2]) folosit de firewall-urile aplicațiilor web (WAF - Web Application Firewall).

În primul rând, sistemul de detecție funcționează în întregime pe baza înregistrărilor realizate de sistemul de operare, și anume jurnalul de autentificare, kernel logs și tabela de conexiuni active (sockets). Astfel, nu este necesară instalarea unui agent pe serviciile monitorizate sau modificarea acestora. Compromisul acestei simplități este dependența de formatul jurnalelor, așa că o eventuală schimbare în modul în care sistemul de operare înregistrează o autentificare eșuată ar necesita ajustarea tiparului corespunzător. Totodată, pentru că fiecare detector ia în calcul doar evenimentele care se încadrează într-o anumită fereastră de timp, scorul prezintă activitatea recentă, în loc să se cumuleze la nesfârșit, fereastra acționând astfel ca



un mecanism de atenuare. O adresă care își încetează comportamentul abuziv iese o dată cu trecerea timpului din atenția sistemului. O direcție de dezvoltare viitoare constă în faptul că toate semnalele sunt calculate cu aceeași pondere, astfel că un comportament frecvent de severitate redusă poate cântări cât unul cu adevărat periculos. Cu o atribuire a unei ponderi diferite fiecărei categorii, scorul ar reflecta într-o măsură mult mai mare severitatea.

### 5.3 Planificatorul adaptiv și logica de decizie

Motorul de detecție doar calculează scorul, decizia rămânând responsabilitatea planificatorului care se execută periodic. La fiecare rulare a acestuia extrage amenințările curente, citește pragurile de avertizare/blocare și indicatorul de blocare automată, urmând să decidă acțiunea luată pentru fiecare adresă IP.

Mecanismul de planificare a fost implementat ca o buclă de interogare ciclică (polling), cu perioadă fixă, în locul unei abordări reactive, bazată pe evenimente, această soluție potrivitându-se în mod natural cu modelul de calculare a scorului pe ferestre de timp folosit de motorul de detecție. Scorul reflectă semnalele acumulate într-un interval, ceea ce corespunde unei evaluări periodice a stării. La fiecare perioadă, sistemul citește jurnalele și conexiunile din interval și le tratează pe toate, împreună. Astfel, ritmul de lucru rămâne legat de frecvența fixă a ciclului, nu de fluxul de evenimente, acesta reprezentând un avantaj real în condiții de atac. Chiar dacă un atacator generează un număr mare de înregistrări în jurnal, ele sunt procesate la un loc, în ciclul care urmează, evitându-se supraîncărcarea aplicației de ritmul atacului. Dacă ar fi folosită o abordare reactivă, efortul sistemului ar crește direct cu intensitatea atacului. Intervalul ales reprezintă un compromis între reactivitate și efort de procesare, fiind destul de scurt încât o decizie de blocare să urmeze atacul aproape imediat, dar suficient de lung cât să fie evitată citirea inutilă a jurnalelor și a tabelii de conexiuni. Figura 10 prezintă pașii unui ciclu de scanare.

Valoarea de bază (baseline) este necesară pentru a preveni blocarea automată, în următorul ciclu, a unei adrese IP deblocate manual, având în vedere că înregistrările încă sunt vizibile în fereastra de detecție. Astfel, atunci când o adresă este deblocată, scorul curent este stocat ca valoare de referință, urmând ca aceasta să fie scăzută din scorul total pentru a reflecta exclusiv activitatea recentă.

Pentru a avea o viteză mai mare de rulare, planificatorul menține starea curentă în memorie, dar aceasta se pierde la repornirea aplicației. Prin urmare, pentru a evita pierderea blocărilor active, starea este inițializată cu datele salvate în baza de date, reîncărcând astfel adresele monitorizate și statusul acestora.

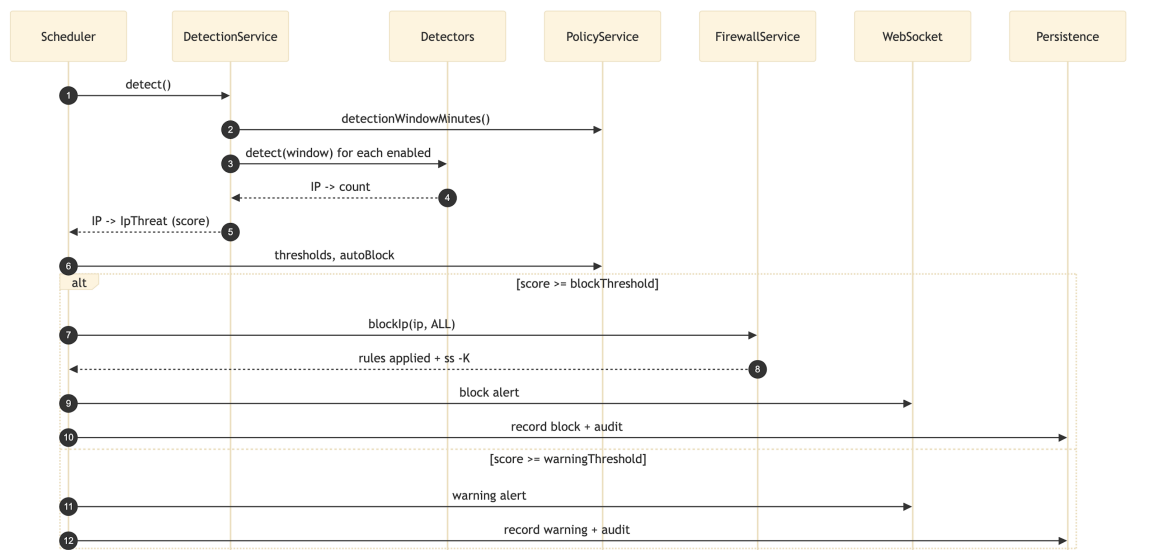


Figura 10: Diagrama de secvență a unui ciclu de scanare

## 5.4 Aplicarea măsurilor pe gazdă

Serviciul de gestionare a firewall-ului este cel care pune deciziile în aplicare, transformând o comandă de blocare în regulile iptables corespunzătoare. O blocare pe toate lanțurile (chains) înseamnă o regulă de tip DROP, pentru adresa specificată, pe lanțurile INPUT, OUTPUT și FORWARD, astfel blocând complet comunicarea cu acea adresă.

A fost aleasă acționarea asupra firewall-ului real de pe sistemul gazdă, în loc de unul simulat, deoarece o protecție efectivă presupune ca regulile să fie aplicate pe gazdă, nu doar simulate. Acoperirea celor trei lanțuri, garantează că o adresă blocată nu poate nici să ajungă la server, nici să primească date de la acesta, și că traficul care doar trece prin gazdă este, de asemenea, oprit. Fiecare dintre aceste comenzi trece printr-un singur serviciu de execuție al comenzilor, singura componentă din sistem care are dreptul de a rula comenzi de sistem. Concentrând astfel toate comenzile, interacționarea cu sistemul de operare, împreună cu privilegiile pe care le presupune, devine ușor de analizat și securizat.

Cu toate acestea, o regulă firewall are efect doar pe noile pachete, care vin după momentul aplicării. Pentru a face ca regula să acționeze instant, să se aplice și pe conexiunile deja stabilite, sistemul închide socket-urile deschise ale adresei IP folosind utilitarul ss, imediat după ce regulile iptables au fost adăugate.

Deblocarea folosește același mecanism, dar execută comanda de ștergere și tratează erorile de tip "regula nu există" provenite de la iptables, returnând succes pentru a menține operația idempotentă.

## 5.5 Port Knocking

Sistemul implementează și mecanismul de *port knocking*, metoda de acces complementară. La pornirea aplicației, un serviciu deschide câte un ascultător TCP pe fiecare port din secvența secretă setată în prealabil, iar fiecare conexiune este înregistrată ca o "bătaie" (knock) pe portul respectiv și transmisă unui automat de stări simplu, ambele componente rulând în backend. Când o adresă completează secvența de porturi în ordinea corectă și în limita de timp permisă, portul protejat (SSH) este deschis pentru client, adăugând o regulă iptables temporară.

Ideea din spatele tehnicii de *port knocking* [13] este aceea că un serviciu poate fi menținut invizibil pentru toată lumea, dar doar cei care cunosc secvența secretă a porturilor pot avea acces la acesta. Doar după ce este contactată ordinea corectă de porturi, firewall-ul lasă, pentru un timp limitat, adresa IP să treacă. Este o formă de securitate menită să completeze autentificarea obișnuită, nu să o înlocuiască. Una dintre condițiile importante este aceea că portul protejat să fie închis în mod implicit. O a doua condiție este existența unei reguli care acceptă traficul conexiunilor deja stabilite (ESTABLISHED), deoarece, altfel, la expirarea ferestrei de acces și regula temporară este eliminată, iar sesiunea deschisă prin knocking ar fi întreruptă, întrucât pachetele sale ar fi din nou respinse.

## 5.6 Politica de securitate configurabilă

Politica este elementul care face sistemul ajustabil în timpul rulării. Aceasta este reprezentată de un singur rând în baza de date, mapat la o entitate *SecurityPolicy*, care stochează pragurile de avertizare/blocare, fereastra de detecție, opțiunea de blocare automată și câte un indicator pentru fiecare detector. Politica este păstrată în cache printr-un serviciu dedicat ce expune metode de acces și salvează modificările primite.

Legătura dintre politica de securitate și motorul de detecție este reprezentată de o singură metodă, care asociază categoria unui detector indicatorului corespunzător, aceasta fiind consultată de orchestrator înainte de a-l rula pe fiecare în parte.

La modificarea politicii de securitate, cererea este validată de către serviciu (pragul de avertizare trebuie să fie mai mic decât cel de blocare), auditează o descriere a modificărilor și reîmprospătează cache-ul. Atât motorul de detecție, cât și scheduler-ul își citesc parametrii din această sursă la fiecare ciclu, astfel că, orice modificare aplicată în interfața web are efect chiar de la următoarea scanare.

Posibilitatea de a edita politica în timpul rulării este ceea ce justifică caracterul adaptiv al sistemului la nivel de configurare. Un administrator poate alege să micșoreze pragurile în timpul unei amenințări sau poate să le relaxeze, să le mărească și poate activa sau dezactiva un anumit serviciu de detecție. Reprezentarea politicii ca un singur rând păstrat în cache este justificată de faptul că gazdei îi corespunde o singură politică și, în plus, citirea ei trebuie să

aibă un cost mic, deoarece se produce la fiecare ciclu. Validarea și auditarea, asociate fiecărei cereri de modificare a politicii, fac schimbarea atât sigură, cât și trasabilă.

## 5.7 Baza de date

Toată starea persistentă se află într-o bază de date PostgreSQL a cărei schemă este construită în jurul a cinci tabele cu ajutorul claselor de entități JPA. Rolurile tabelor sunt rezumate în Tabela 4, iar relațiile dintre ele sunt ilustrate în Figura 16.

Tabela 4: Tabelele și rolurile acestora

| Tabelă                 | Rol                                                                      |
|------------------------|--------------------------------------------------------------------------|
| <b>tracked_ip</b>      | Înregistrarea principală pentru fiecare adresă observată, starea curentă |
| <b>block_record</b>    | Istoricul fiecărei blocări și starea ei activă                           |
| <b>audit_log</b>       | Istoricul tuturor acțiunilor executate                                   |
| <b>ip_whitelist</b>    | Lista adreselor de încredere                                             |
| <b>security_policy</b> | Politica de securitate                                                   |

Tabela *tracked\_ip* conține pe fiecare rând adresa IP, starea curentă a acesteia (WARNING, BLOCKED sau CLEARED), numărul de semnale suspecte însumate, valoarea de referință (baseline) folosită pentru a împiedica reblocarea imediată după o deblocare, precum și momentele primei și ultimei observări a adresei.

În *block\_record* este reținut fiecare eveniment de blocare. O adresă poate fi blocată, deblocată și blocată din nou, fiecare acțiune reprezentând o înregistrare separată în tabelă, ceea ce permite reconstituirea întregului istoric al unei adrese. Fiecare înregistrare conține o cheie externă (foreign key) către adresa urmărită din tabela descrisă anterior, o stare (ACTIVE, EXPIRED sau REMOVED), lanțul pe care a fost aplicată, sursa deciziei (AUTO pentru scheduler sau MANUAL pentru un administrator), un motiv și momentele relevante. Această tabelă este indexată după stare și cheia externă a adresei, coloanele după care se interoghează atunci când se caută blocările active.

Evidența fiecărei acțiuni efectuate în sistem este ținută în *audit\_log*, fiind o tabelă în care doar se adaugă (append-only). Fiecare intrare stochează acțiunea, tipul de actor (USER sau SYSTEM), identitatea actorului în cazul utilizatorilor, o referință opțională către adresa vizată, deoarece există și acțiuni ce nu țin cont de o adresă IP, detalii și momentul creării. Un aspect important este că identificatorul actorului este stocat ca subiectul din Keycloak, nu ca o cheie externă într-o tabelă locală de utilizatori, deoarece utilizatorii există în Keycloak, nu în aplicație. Această tabelă este indexată după momentul creării, folosit la ordonarea evenimentelor în pagina de istoric, și după acțiune, care este folosită la gruparea în analiză.

Lista cu adrese de încredere este stocată în *ip\_whitelist*, și conține adresa, o notă opțională, cine a adăugat-o și când. Această listă este consultată de către planificator, iar în cazul în

care o adresă se regăsește aici, se trece peste aceasta.

Politica de securitate este reprezentată de o singură înregistrare din tabela *security\_policy*.

Păstrarea acestor entități în tabele separate, în loc de a încărca una singură, este ceea ce permite ca aceleași evenimente să alimenteze, în paralel, starea în timp real din dashboard, gestionarea blocărilor și paginile de istoric și de analiză, fără ca vreuna dintre ele să le constrângă pe celelalte.

Pe lângă structura propriu-zisă, modul de stocare este completat de câteva detalii. Enumerările (enum) sunt salvate ca text, nu ca indici numerici, ceea ce face ca baza de date să rămână lizibilă și previne coruperea intrărilor existente la o eventuală reordonare a constantelor în cod. De asemenea, pentru a asigura o sincronizare perfectă între logica aplicației și structura bazei de date, s-a utilizat tehnica ORM, iar schema bazei de date este generată și actualizată automat la pornirea sistemului pe baza claselor de entități. Totodată, sistemul include un mecanism care inserează automat o politică de securitate implicită dacă tabela corespunzătoare este goală.

## 5.8 Autentificare și controlul accesului

Backend-ul aplicației este un server de resurse OAuth2 [15] fără stare (stateless). Acesta nu gestionează nici sesiuni, nici parole, dar fiecare cerere trebuie să poarte un jeton (token) JWT (JSON Web Token) [16] valid, emis de Keycloak, serverul de autorizare, pe care configurația de securitate îl validează.

Autorizarea se realizează printr-un control al accesului bazat pe roluri. Rolurile definite în Keycloak sunt transportate în interiorul token-ului și sunt preluate din proprietatea *realm\_access*, apoi convertite în permisiuni specifice Spring Security (authorities).

Pe baza acestor roluri, securitatea la nivel de metodă este punctul de decizie, unde se stabilește ce tip de utilizator are acces la fiecare endpoint.

Aceeași separare se regăsește și în interfața grafică, unde mecanismele de protecție ale rutelor (route guards) nu permit utilizatorilor fără rol de administrator să acceseze paginile de firewall, politică și istoric. Aceste mecanisme însă nu sunt de ajuns, chiar dacă paginile sunt ascunse și inaccesibile unui utilizator obișnuit, acestea ar putea fi ocolite printr-un apel direct la API. Astfel punctul de securitate real rămâne verificarea de pe server, aplicată pe fiecare endpoint sensibil.

La baza acestui design stă caracterul stateless al aplicației, backend-ul nepăstrând nicio stare, ci evaluând fiecare cerere pe baza token-ului atașat, fapt care elimină nevoia unei stocări a sesiunilor pe server și menține API-ul simplu.

Gestionarea identității, care cuprinde parolele, emiterea token-urilor, utilizatorii și rolurile, nu a trebuit să fie implementată deoarece această parte a fost desemnată integral către un serviciu

extern specializat, Keycloak.

## 5.9 Notificări în timp real

Pentru ca administratorul să vadă evenimentele produse, backend-ul trimite notificările printr-o conexiune WebSocket, folosind protocolul STOMP (Simple Text Oriented Message Protocol), clienții abonându-se la un canal (topic), unde serverul publică câte un eveniment ori de câte ori este executată o acțiune de avertizare, blocare sau deschidere temporară a unui port pentru o adresă.

Fiecare alertă conține adresa, tipul evenimentului, numărul de semnale și un mesaj, iar interfața o prezintă în momentul producerii.

Alegerea utilizării unui canal WebSocket a fost una potrivită aici deoarece evenimentele sunt generate de server, nu cerute de către client. O abordare de interogare repetată a unui endpoint ar rămâne în urmă față de evenimentele deja produse sau ar irosi cereri atunci când nu s-au produs modificări. Difuzarea pe un singur canal este suficientă, fluxul de evenimente de securitate fiind comun pentru toți utilizatorii. Prin el trec cele trei tipuri de alerte, fiecare fiind afișat în mod corespunzător.

## 5.10 Monitorizarea rețelei

Pagina de monitorizare (Figura 11) prezintă două tabele de tip doar citire (read only) de pe sistemul gazdă, și anume conexiunile active și regulile de firewall curente, ambele fiind extrase cu ajutorul unui serviciu de monitorizare care rulează comenzi specifice sistemului de operare și prelucrează rezultatul în obiecte structurate care pot fi afișate în interfața web. Datele despre conexiuni sunt generate de utilitarul *ss*, iar cele de firewall de *iptables*, cele din urmă necesitând un proces mai complex, rezultatul comenzii e grupat pe lanțuri (chains), iar coloanele cu informații trebuie extrase dintr-un text neformatat.

Prelucrarea rezultatelor generate de comenzile native ale sistemului este o abordare fragilă deoarece depinde de formatul coloanelor, iar o versiune diferită a utilitarului ar putea modifica formatul acestora. Prin urmare, logica de extragere a fost concentrată într-un singur serviciu, restul sistemului interacționând doar cu datele deja prelucrate de acest serviciu. O dată ce informația a fost structurată, interfața o poate pagina și afișa administratorului, care mai departe poate efectua o căutare sau filtrare, aceasta fiind necesară deoarece o gazdă solicitată poate avea un număr mare de conexiuni și reguli.

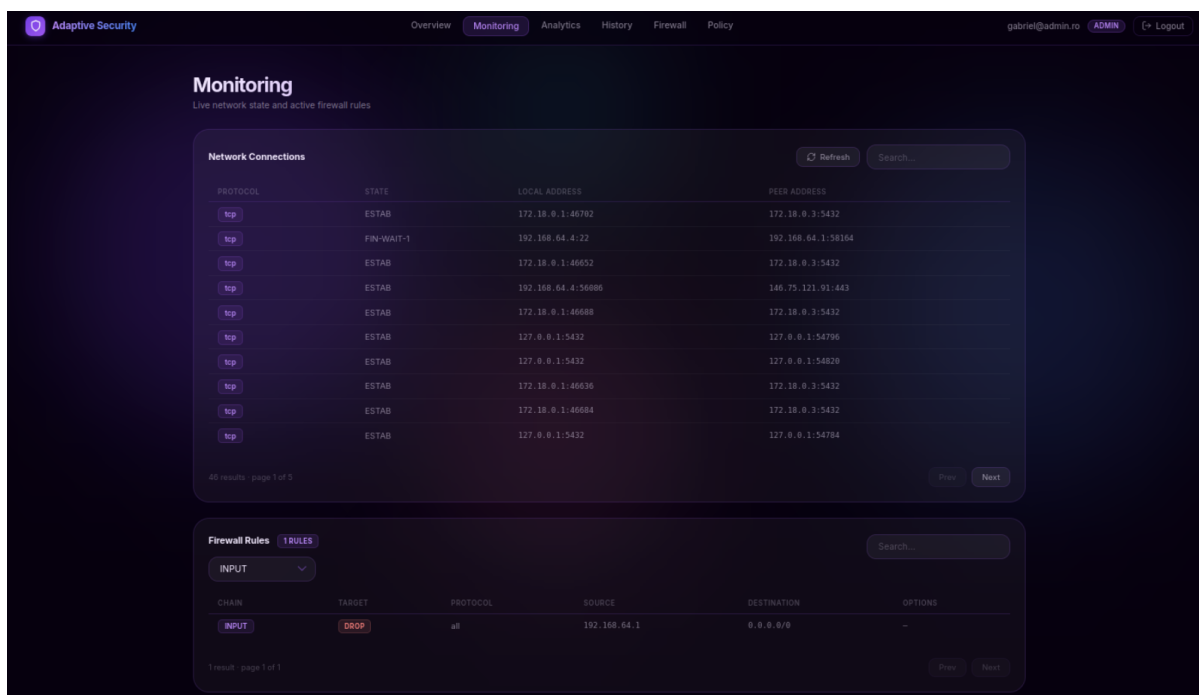


Figura 11: Pagina de monitorizare

## 5.11 Asistentul AI

Asistentul permite unui administrator să exprime acțiunea pe care dorește să o execute în limbaj natural (spre exemplu: "blochează tot traficul din această subrețea"), urmând ca aceasta să fie interpretată ca o acțiune efectivă de securitate. Aceasta îi este propusă administratorului, care va trebui să o valideze pentru a fi aplicată.

Pentru a obține un răspuns previzibil, cererea către API-ul Gemini folosește un mecanism de ieșire structurată (structured output) [4], modelul returnând un răspuns în format JSON (JavaScript Object Notation), alcătuit dintr-o acțiune, o adresă, un lanț și o notă. Mai mult decât atât, modelul primește un prompt care restrânge acțiunile pe care le poate propune agentul, iar temperatura este setată la zero, pentru a primi un răspuns determinist. Rezultatul obținut poate fi prelucrat și verificat, astfel, direct de cod fără a mai fi nevoie de interpretarea acestuia ca text liber.

Înainte de prezentarea răspunsului ca o sugestie de acțiune executabilă, acesta este validat pe server, acțiunile necunoscute, adresele incorecte și lanțurile invalide fiind respinse. Un rezultat eronat sau neașteptat al modelului nu poate deveni o comandă care să fie aplicată.

Asistentul acoperă cele trei acțiuni pe care sistemul le poate executa deja, și anume cea de blocare, deblocare și adăugare în lista de încredere. Propunerile confirmate sunt executate prin aceleași endpoint-uri folosite și la aplicarea manuală, iar înainte de confirmare, administratorul poate inspecta comanda efectivă din spatele fiecărei sugestii generate cu inteligență artificială.

## 5.12 Interfața web și pagina de analiză

Interfața web a fost realizată ca o aplicație Angular, care afișează în timp real starea sistemului și alertele primite de la backend (Figura 12).

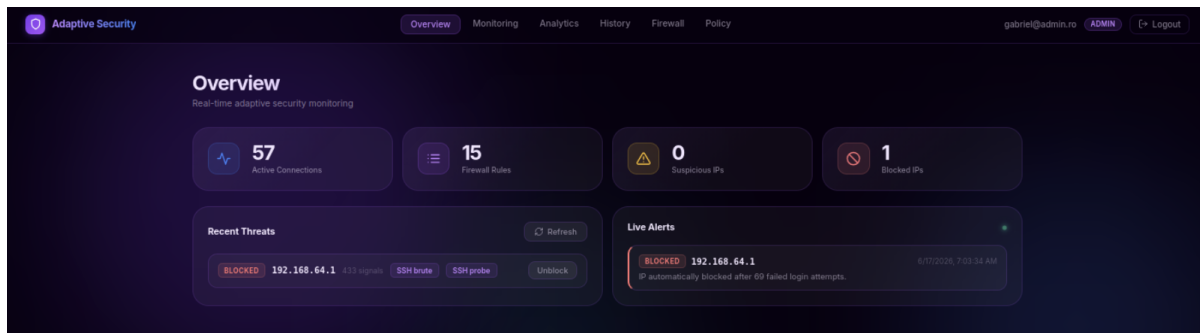


Figura 12: Dashboard

O abordare ce oferă o detecție a modificărilor mai explicită și mai eficientă, este rularea aplicației fără *zone.js* (zoneless). Angular nu mai interceptează operațiunile asincrone pentru a reîmprospăta automat interfața, astfel că detecția modificărilor este apelată explicit după fiecare acțiune ce schimbă starea. Întreaga comunicare cu backend-ul este organizată în servicii, ușor de gestionat.

Secțiunea de analiză (Figura 13) a fost concepută pentru a oferi o imagine de ansamblu clară asupra evenimentelor din sistem din anumite perioade. Jurnalul de audit și adresele monitorizate sunt centralizate și transformate în statistici, care includ numărul de acțiuni pentru fiecare tip, adresele cele mai active și evoluția zilnică a avertizărilor și blocărilor.

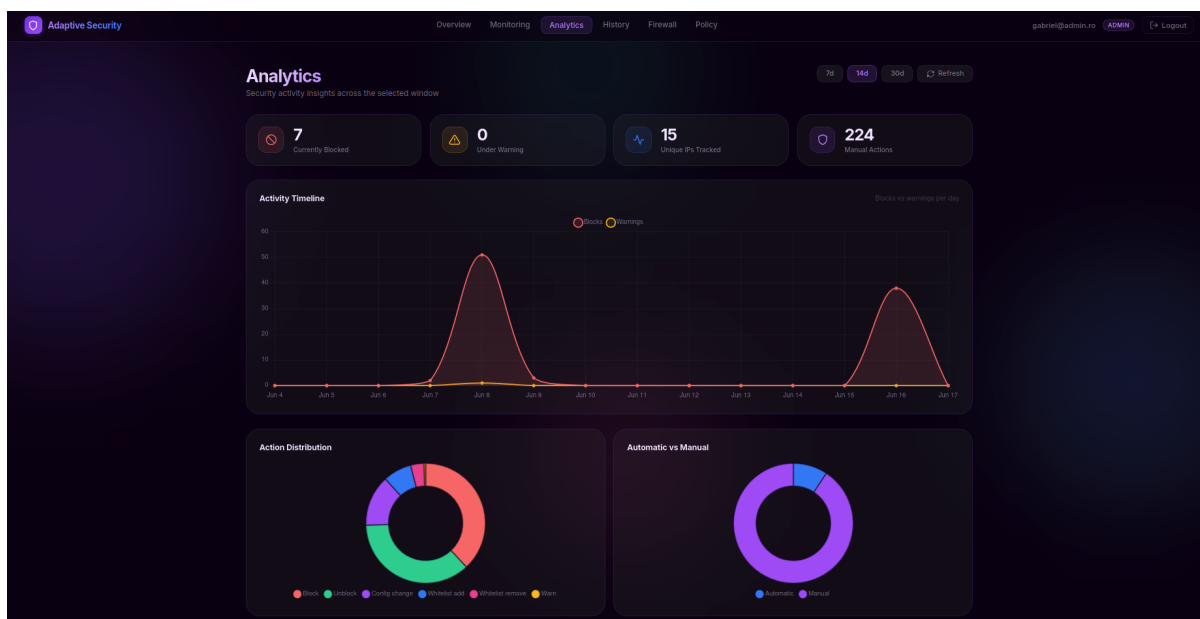


Figura 13: Pagina de analiză



## 6 EVALUARE

### 6.1 Verificarea funcționalităților

Corectitudinea aplicației a fost verificată pe trei niveluri. La nivelul componentelor au fost scrise teste unitare pentru a valida logica de bază, izolată de restul aplicației. La nivelul API-ului, rutele expuse de către server au fost testate cu ajutorul Swagger UI, generat automat din specificația OpenAPI a aplicației (Figura 14). La testarea aplicației în ansamblu a fost utilizată testarea de tip black-box, verificând cum se comportă aplicația, privită din afară, fără a recurge la structura codului. Au fost reproduse atacuri reale prin scripturi rulate de pe mașina locală, precum autentificări SSH repetate sau deschiderea unui număr mare de conexiuni. Pe lângă reproducerea de atacuri, de pe mașina locală a fost testată și funcționalitatea de port knocking, prin trimitere de cereri de conectare în ordinea corectă a porturilor prestabilite și verificarea faptului că portul SSH se deschide temporar pentru adresa IP a mașinii de pe care au fost inițiate conexiunile.

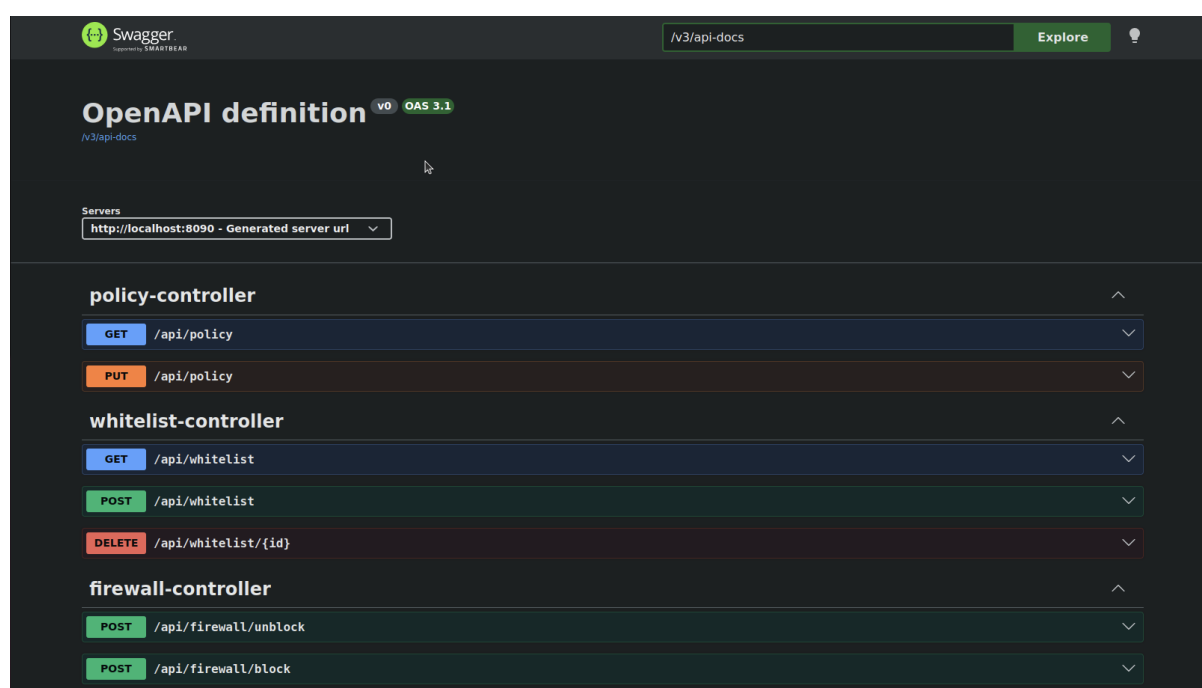


Figura 14: Swagger UI

Testarea a pornit de la stratul de autentificare, care stă la baza securității aplicației. A fost verificat faptul că doar cererile care au în componența lor un token valid sunt acceptate de către server și ca rolurile să fie respectate, un utilizator de tip implicit având dreptul de a consulta starea sistemului, iar administratorul să poată executa acțiuni privilegiate. În aceste

verificări au fost integrate și situații de eroare, cereri fără token sau cu unul invalid, pentru a confirma că aplicația refuză cererile neautorizate.

Fluxul central, cel prin care o adresă atacatoare este detectată și, mai apoi, oprită, a fost testat cel mai detaliat. S-au folosit scripturi de atac și a fost urmărit parcursul unei surse, de la jurnalele ce înregistrează evenimentele până la blocarea efectivă prin adăugarea de reguli firewall și închiderea eventualelor conexiuni active. A fost verificat faptul că adresa este marcată mai întâi ca suspectă, afișând doar un avertisment pentru aceasta, iar în ultimă instanță, la depășirea pragului, că aceasta a fost blocată.

De asemenea, au fost testate și intervențiile manuale și configurațiile administratorului, cum ar fi blocarea sau deblocarea unei adrese, adăugarea sau eliminarea unei adrese din lista de încredere, modificarea pragurilor și activarea sau dezactivarea detectoarelor, verificând că modificările de politică au efect imediat, la următorul ciclu. S-a validat și faptul că o adresă deblocată manual nu este imediat blocată, evidențiind faptul că administratorul are control deplin și decizia finală asupra sistemului. Funcționalitatea de asistență a fost verificată atât din punct de vedere al corectitudinii cererii, cu unele valide de tipul "Blochează adresa 192.168.62.1", dar și cu cereri invalide precum "Cum este vremea astăzi?", pe cea din urmă respingând-o, cât și din punct de vedere al execuției, acțiunea fiind doar propusă spre confirmare, nu și executată.

Testele finale au însemnat verificarea paginilor care expun date de pe sistemul gazdă și statisticile generate de aplicație, confirmând corectitudinea informațiilor returnate și conformitatea acestora cu starea reală a sistemului.

## 6.2 Evaluarea performanțelor

În plus față de testarea funcțională a rutelor, au fost mășurați și timpii de răspuns ai principalelor endpoint-uri ale interfeței REST, fiecare dintre acestea fiind interogată în mod repetat. Tabela 5 prezintă valoarea minimă, maximă și media duratei fiecărui răspuns.

Tabela 5: Timpii de răspuns ai endpoint-urilor

| Endpoint                       | Minim(ms) | Medie(ms) | Maxim(ms) |
|--------------------------------|-----------|-----------|-----------|
| /api/monitoring/connections    | 1.5       | 8.7       | 20.2      |
| /api/monitoring/firewall-rules | 2.9       | 10.5      | 47.0      |
| /api/monitoring/suspicious-ips | 1.1       | 4.2       | 8.5       |
| /api/analytics/overview        | 1.6       | 7.0       | 13.7      |
| /api/events?page=0&size=20     | 1.2       | 6.7       | 13.3      |
| /api/policy                    | 1.0       | 7.4       | 13.4      |
| /api/whitelist                 | 1.1       | 5.3       | 8.3       |

Toate rutele răspund rapid, într-un timp de ordinul milisecundelor, fapt care indică o interfață promptă. Se observă că rutele care aduc informații din baza de date a aplicației au timpi de

răspuns mai buni decât cele care interoghează sistemul de operare, din cauza pornirii unui proces extern.

De asemenea, a fost evaluat și consumul de resurse ale aplicației, prin două situații diferite, una în care sistemul este atacat și una în stare normală, de repaus. Atacul a fost făcut printr-un script rulat pe mașina locală către mașina virtuală pe care rulează aplicația, declanșând astfel mecanismul de blocare. Consumul de procesor și memorie a fost eșantionat la fiecare secundă timp de 30 de secunde, pentru fiecare dintre cele două situații, de mai multe ori.

Memoria a fost extrasă ca memorie rezidentă (RSS), fiind porțiunea exactă de memorie fizică (RAM) pe care procesul o folosește.

Consumul de procesor a fost calculat folosind formula<sup>1</sup>, unde *processTime* este timpul CPU consumat în intervalul de eșantionare, iar *elapsedTime* durata intervalului:

$$CPU = \frac{processTime}{elapsedTime} \times 100 \quad (1)$$

Rezultatele din Tabela 6 arată o folosire a resurselor redusă, memoria rămânând stabilă la aproximativ 410 MB în ambele situații, ceea ce a fost așteptat deoarece aplicația nu procesează direct traficul, ci analizează periodic jurnalele.

Consumul de procesor este în general neglijabil, media fiind în repaus sub 1%, dar cu vârfuri scurte în timpul unui atac. Aceste vârfuri apar în momentul în care o adresă este efectiv blocată, detectarea în sine nereprezentând o sarcină prea costisitoare.

Tabela 6: Consumul de resurse

| Rulare          | RAM(MB) | CPU medie (%) | CPU maxim(%) |
|-----------------|---------|---------------|--------------|
| <b>Repaus 1</b> | 411     | 0.5           | 3            |
| <b>Repaus 2</b> | 410     | 0.2           | 2            |
| <b>Repaus 3</b> | 408     | 0.3           | 2            |
| <b>Atac 1</b>   | 411     | 1.1           | 16           |
| <b>Atac 2</b>   | 408     | 1.7           | 31           |
| <b>Atac 3</b>   | 410     | 1.0           | 9            |

### 6.3 Discuție

În urma evaluării s-a confirmat că soluția respectă cerințele definite în al doilea capitol. Fluxul central a fost testat și validat cap-la-cap (E2E), de la înregistrarea evenimentelor la închiderea conexiunilor active.

Din punct de vedere al performanței, măsurătorile indică un consum redus de resurse, interfața răspunde prompt, iar memoria rămâne stabilă indiferent de situația în care se află.

<sup>1</sup><https://man7.org/linux/man-pages/man1/ps.1.html>

Cea mai relevantă comparație este cea cu fail2ban, fiind instrumentul cel mai apropiat ca scop, ambele măsurate și rulate pe aceeași mașină, în aceleași condiții, folosind același script de atac. Fail2ban are un consum de memorie mai mic (circa 36MB), față de aproximativ 410MB, cât folosește soluția propusă, iar consumul de procesor este neglijabil la ambele.

Deși fail2ban consumă mai puține resurse, acesta oferă detalii minimale despre atacuri, neavând o interfață dedicată, alerte în timp real, statistici despre activitatea sistemului sau un istoric al acțiunilor efectuate.

## 7 CONCLUZII

Lucrarea de față a pornit de la problema atacurilor constante către servere Linux expuse în internet. Soluțiile care se găsesc pe piață sunt adesea fie limitate din punct de vedere al monitorizării și al informațiilor oferite asupra stării sistemului, fie prea complexe și costisitoare pentru un utilizator individual. Obiectivul lucrării a fost realizarea unui sistem de securitate adaptiv care să detecteze automat sursele cu comportament suspect și să reacționeze prompt prin blocarea acestora, oferind vizibilitate și control deplin asupra întregului proces.

Soluția propusă monitorizează periodic jurnalele sistemului de operare și starea conexiunilor, unificând mai multe tipuri de evenimente într-un scor unic atribuit fiecărei adrese IP. La depășirea unor praguri configurabile sistemul avertizează și ulterior blochează adresa prin reguli de firewall aplicate la nivelul gazdei. Aceste capacități sunt reunite într-o singură aplicație, în locul unui ansamblu de utilitare separate, totul fiind administrabil dintr-o interfață web.

Evaluarea a confirmat că soluția funcționează respectând cerințele și că o face cu un consum de resurse redus. Comparția cu soluțiile existente poziționează aplicația între instrumente minimale precum fail2ban și sistemele complexe de tip SIEM precum Wazuh. Procesul de evaluare a contribuit la îmbunătățirea soluției, evidențiind situații tratate incorect, care au fost remediate ulterior.

Proiectul își atinge obiectivele propuse și demonstrează că un sistem de blocare adaptiv constituie o alternativă potrivită pentru protejarea unui server individual, oferind administratorului o reacție automată și o imagine clară asupra stării sistemului.

### 7.1 Dezvoltări ulterioare

Soluția poate fi extinsă în mai multe direcții. O primă direcție este extinderea detecției către traficul HTTP. Prin interceptarea și analiza cererilor primite de aplicațiile web care rulează pe server, sistemul ar putea identifica și bloca atacurile la nivelul aplicației, nu doar pe cele vizibile în jurnalele sistemului de operare.

O a doua direcție este scalarea soluției către mai multe servere. În prezent, o instanță poate proteja un singur server, astfel că o dezvoltare utilă ar fi posibilitatea de a administra mai multe servere dintr-un punct central, printr-o consolă unică. În acest context ar putea fi implementat un mecanism de partajare a unei liste negre (blacklist) între serverele coordonate, astfel încât o adresă blocată pe un server, va fi blocată preventiv și pe celelalte înainte de a deveni o amenințare, valorificând astfel informațiile colectate pe fiecare gazdă și contribuind la creșterea nivelului de protecție al infrastructurii.

## BIBLIOGRAFIE

- [1] Angular vs React vs Vue: Core Differences. <https://www.browserstack.com/guide/angular-vs-react-vs-vue>. Accesat: Iunie 2026.
- [2] Anomaly Scoring. [https://coreruleset.org/docs/2-how-crs-works/2-1-anomaly\\_scoring/](https://coreruleset.org/docs/2-how-crs-works/2-1-anomaly_scoring/). Accesat: Iunie 2026.
- [3] Fail2ban. <https://github.com/fail2ban/fail2ban>. Accesat: Iunie 2026.
- [4] Gemini - Structured Outputs. <https://ai.google.dev/gemini-api/docs/structured-output#rest>. Accesat: Iunie 2026.
- [5] Local API. [https://docs.crowdsec.net/u/user\\_guides/lapi\\_mgmt](https://docs.crowdsec.net/u/user_guides/lapi_mgmt). Accesat: Iunie 2026.
- [6] PostgreSQL. <https://www.postgresql.org/about/>. Accesat: Iunie 2026.
- [7] React. <https://react.dev/>. Accesat: Iunie 2026.
- [8] Spring Boot. <https://spring.io/projects/spring-boot>. Accesat: Iunie 2026.
- [9] Wazuh. <https://documentation.wazuh.com/current/getting-started/index.html>. Accesat: Iunie 2026.
- [10] P. Bischoff. Hackers attack 70 times per minute: what happens when a computer is left unsecured on the internet. <https://www.comparitech.com/blog/information-security/honeypot-computer-study/>. Accesat: Iunie 2026.
- [11] J. Chen. Observing attacks against hundreds of exposed services in public clouds. <https://unit42.paloaltonetworks.com/exposed-services-public-clouds/>. Accesat: Iunie 2026.
- [12] Cisco. Observations of login activity in an ssh honeypot. [https://sec.cloudapps.cisco.com/security/center/resources/ssh\\_honeypot.html](https://sec.cloudapps.cisco.com/security/center/resources/ssh_honeypot.html). Accesat: Iunie 2026.
- [13] R. deGraaf, J. Aycock, and M. Jacobson. Improved port knocking with strong authentication. In *Proc. 21st Annual Computer Security Applications Conference (ACSAC, Tucson, AZ, December 2005)*, 2005. Accesat: Iunie 2026.
- [14] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

- [15] IETF. RFC 6749: The OAuth 2.0 Authorization Framework. <https://datatracker.ietf.org/doc/html/rfc6749>, 2012. Accesat: Iunie 2026.
- [16] IETF. RFC 7519: JSON Web Token (JWT). <https://datatracker.ietf.org/doc/html/rfc7519>, 2015. Accesat: Iunie 2026.
- [17] MongoDB. What is MongoDB? <https://www.mongodb.com/docs/manual/>. Accesat: Iunie 2026.
- [18] Netfilter. The nftables project. <https://www.netfilter.org/projects/nftables/index.html>. Accesat: Iunie 2026.
- [19] OISF. Suricata - features. <https://suricata.io/features/>. Accesat: Iunie 2026.
- [20] OWASP. ModSecurity Project. <https://modsecurity.org>. Accesat: Iunie 2026.
- [21] J. Owens and J. Matthews. A study of passwords and methods used in brute-force ssh attacks. In *USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET)*, 2008.
- [22] R.S. Sandhu, E.J. Coyne, H.L. Feinstein, and C.E. Youman. Role-based access control models. *Computer*, 29(2):38–47, 1996.
- [23] Stack Overflow. Most Popular Technologies. <https://survey.stackoverflow.co/2025/technology>. Accesat: Iunie 2026.

## ANEXE

### Diagrame

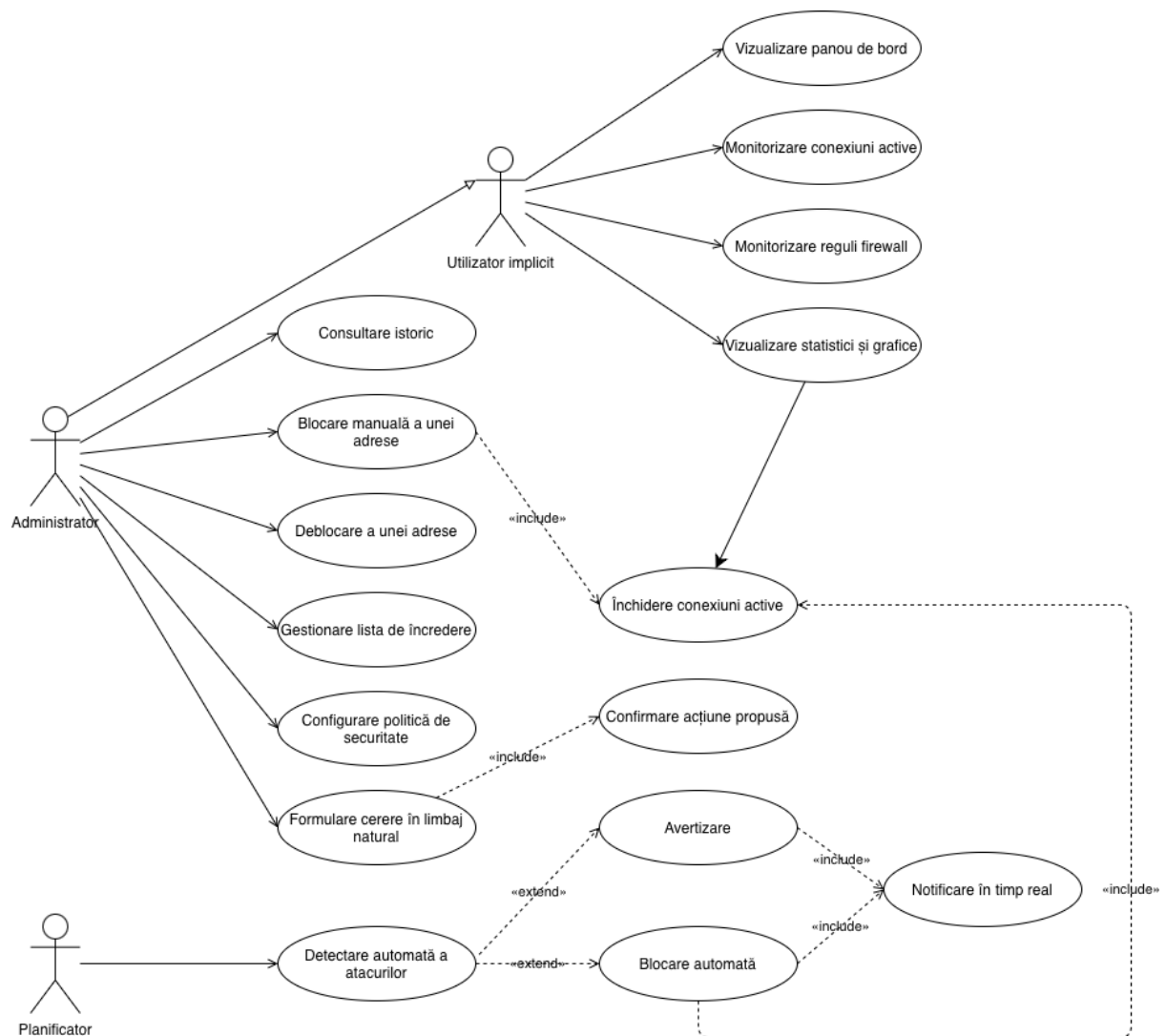


Figura 15: Cazuri de utilizare

...



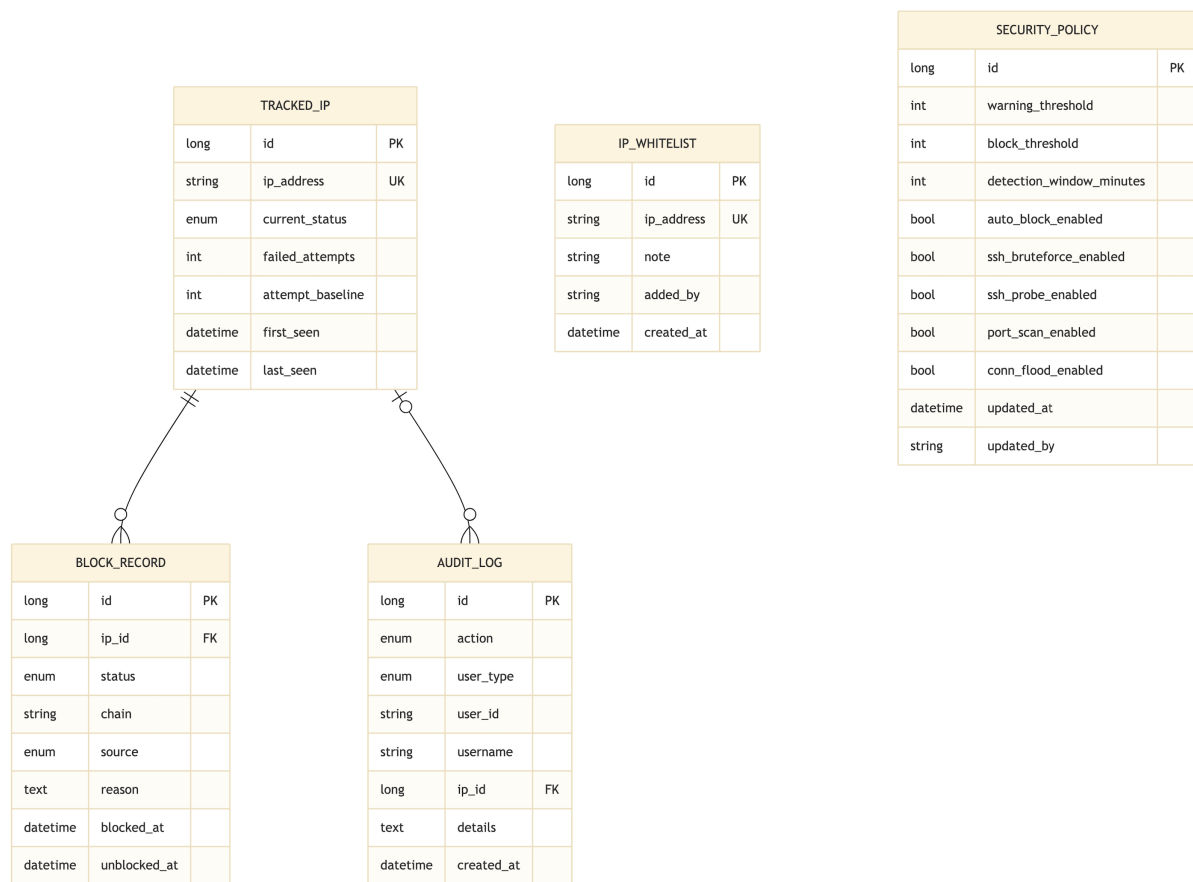


Figura 16: Diagrama entitate-relație a bazei de date